

UNIDAD 2: Acceso a base de datos relacionales

Índice

1.	Acceso a bases de datos relacionais utilizando JDBC.....	2
1.1	Protocolos de acceso a base de datos relacionais.....	2
1.2	API'S de acceso a base de datos	3
1.3	APIS standards para acceso a base de datos	5
2.	Base de Datos Independentes e Embebidas.....	7
2.1	Base de datos independientes.....	7
2.2	Bases de datos embebidas.....	8
3.	Aceso a base de datos relacionais dende Java: Conectividad JDBC.....	9
3.1	Modelos de acceso da API JDBC ás base de datos	10
3.2	Tipos de Conectores JDBC	11
4.	Paquetes da API JDBC	14
4.1	Clases e interfaces do paquete java.sql	15
4.2	Pasos para acceder a unha base de datos utilizando API JDBC	16
4.3	O interface Connection.....	20
4.3.1	Crear e executar instrucións SQL.....	21
5.	Execución de sentenzas de consulta e modificación de datos e procedementos almacenados.....	24
5.1	Execución de sentenzas SQL simples: A interface Statement.....	25
5.2	Execución de sentenzas precompiladas: A interface PreparedStatement	31
5.3	Executar procedementos almacenados e funcións: Interface CallableStatement.....	34
6.	Acceso aos Metadatos.....	40
6.1	Interface DatabaseMetaData.....	40
6.2	Interface ResultSetMetaData.....	50
7.	Xestión de Transaccións e procesamento por lotes.....	52
7.1	Xestión de transaccións	52
7.1.1	Puntos de retorno (Savepoints)	61
7.2	Procesamento por lotes e transaccións.....	63

1. Acceso a bases de datos relacionais utilizando JDBC.

1.1 Protocolos de acceso a base de datos relacionais.

A maioría dos SXBD relacionais teñen **unha interface interactiva** para introducir e executar directamente as ordes SQL.

Exemplo: en ORACLE, a utilidade para este fin chámase SQL*Plus. En SQL Server, ten unha ferramenta gráfica chamada SQL Server Management Studio (SSMS).

Tamén existen ferramentas web modernas como phpMyAdmin (MySQL/MariaDB), pgAdmin (PostgreSQL) ou SQL Developer Web (Oracle)

A interface interactiva é conveniente para crear a base de datos e as táboas, restricións ou para realizar consultas ad hoc ocasionais. Isto implica que o sistema permite ao usuario personalizar unha consulta en tempo real, sen ter que estar atado ás consultas prediseñadas das aplicacións.

Con todo, a maior parte **dnas interaccións coas bases de datos fanse a través de programas de aplicacións** que presentan, introducen e actualizan datos na base de datos relacional. Estes programas coñécense como **aplicacións de base de datos**.

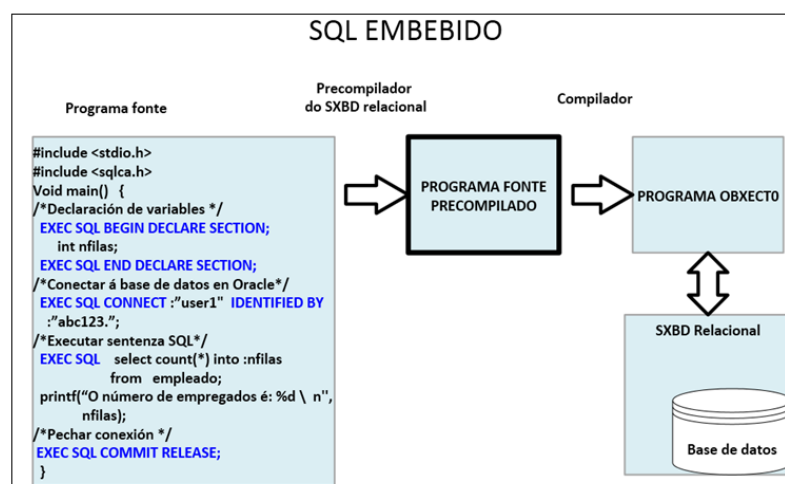
Existen **varias aproximacións para interactuar cunha base de datos dende un programa de aplicación:**

▪ **Embeber as ordes SQL** dunha base de datos relacional *nunha linguaxe de programación anfitrión* (LPA) de propósito xeral (C, Pascal, etc.). As ordes identifícanse por un prefixo, por exemplo:

- Conectar a unha base de datos: EXEC SQL CONNECT
- Declarar variables: EXEC SQL BEGIN/END DECLARE SECTION
- Executar sentenzas SQL: EXEC SQL Sentenza.

As consultas SQL, como linguaxe declarativa, poden devolver unha relación (un conxunto de rexistros). As linguaxes procedimentais non conteñen habitualmente unha estrutura de datos apropiada para iso, xa que logo, SQL proporciona un mecanismo de adaptación coñecido como *cursor* para ir accedendo aos rexistros individuais.

Necesítase un **precompilador** que prové o vendedor do SGBD (en Oracle, por exemplo: Prol*C) que traduce as sentenzas SQL converténdooas en chamadas a unhas librerías específicas, polo tanto, **é dependente dun SXBD relacional en particular**.

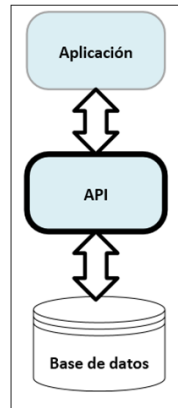


Exemplo do proceso de compilación dun programa con sentenzas SQL embebidas na linguaxe C

✓ Esta técnica está **en desuso en contornos modernos**.

- ✓ Require precompiladores específicos e é **dependente do SXBD**, o que **penaliza a portabilidade**.
- ✓ Foi substituída por **APIs estándar** e frameworks que permiten maior independencia e mantemento.

- **Usar unha biblioteca de funcións de base de datos.** En lugar de modificar os compiladores ou empregar precompiladores, pódense utilizar **librerías de funcións de acceso á base de datos**. Esta aproximación proporciona, o que se coñece como **API (Application Programming Interface)** para acceder a base de datos. As aplicacións só se comunican coa API e esta é a encargada de comunicarse coa base de datos.



Exemplo de Acceso a base de datos utilizando unha biblioteca de funcións

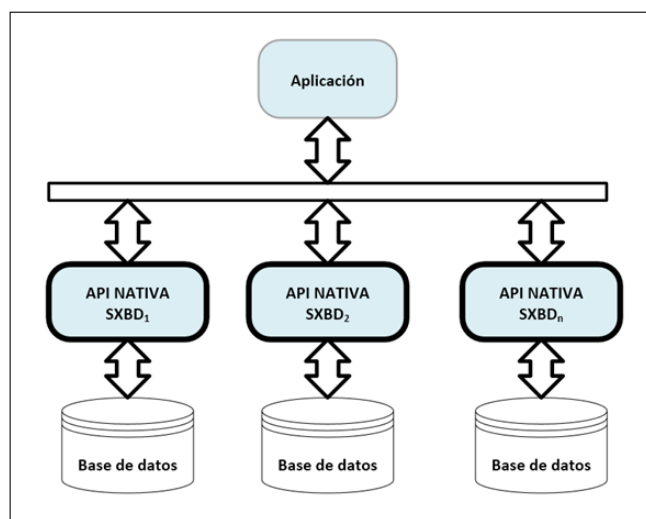
1.2 API'S de acceso a base de datos

Unha **API de acceso a base de datos** é **unha biblioteca de funcións e interfaces para realizar a maioría de accións de manexo de base de datos que os programadores poden requirir**: conectar cunha base de datos, executar sentenzas SQL, recuperar unha fila individual do resultado dunha consulta, etc.

Para poder acceder e manipular a información dunha base de datos é necesario levar a cabo a instalación de certas APIs, que son indispensables para efectuar a conectividade dos datos externos, e vincularlas á aplicación para a súa correcta e adecuada utilización.

Existen varias API de acceso a base de datos:

- **APIs nativas:** a maioría dos fabricantes de SXBD ofrecen APIs nativas propias para acceder as súas bases de datos e que son únicas.



Estrutura xeral de acceso a distintas bases de datos utilizando APIs Nativas de cada fabricante

- ✓ **Vantaxe:** están **optimizadas polo fabricante** e dan soporte completo ás funcionalidades específicas do SXBD. A conexión é directa e non precisa de intermediarios.

- ✓ **Inconveniente:** son **diferentes para cada SXBD**. Cambian os nomes das funcións, o tratamento de erros e o formato dos resultados, polo que o código **non é portable entre diferentes bases de datos**.

Un exemplo de API nativa é Oracle Call Interface (OCI). Esta interface nativa permite conectar e procesar sentencias SQL nunha base de datos Oracle sen necesidade de ningún controlador externo. Para utilizar a API OCI, so se necesita instalar e configurar as librarías cliente (como **Oracle Net Services**) nas computadoras clientes para acceder ao servidor Oracle.

- **APIs Standard de acceso a base de datos:** Están compostas por un **conxunto de bibliotecas** de funcións que **unifican e estandarizan o acceso ás bases de datos**.

Grazas a isto, *unha aplicación pode acceder a calquera base de datos sen necesidade de modificar o código*

Para que isto sexa posible, **cada fabricante debe proporcionar un conector (driver)** que implemente a especificación da API estándar. A aplicación realiza as chamadas ao driver, e este traduce as peticións ás ordes específicas do SXBD.

Conector ou driver

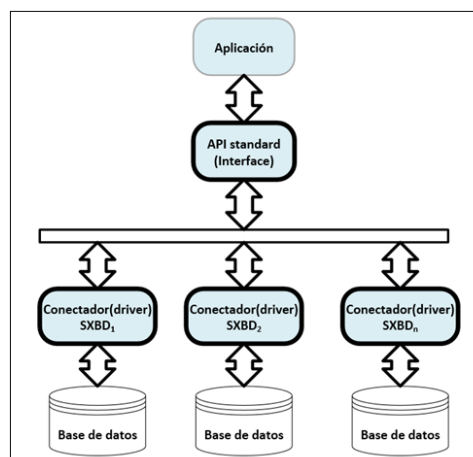
Conector ou driver: **Conjunto de clases** que implementan **las interfaces de la API** y permiten **comunicarse con un SXBD concreto**.

Cada conector é **específico para un SXBD en particular**, por exemplo, un conector para Oracle non pode acceder directamente aos datos do SXBD MySQL.

Limitaciones: Solo puede implementar lo que el SXBD soporte (por ejemplo, si no hay soporte para combinaciones externas, el driver tampoco lo permitirá).

As **tarefas específicas** realizadas polos conectores son:

- ✓ Conexión e desconexión da base de datos.
- ✓ Envío das instrucións SQL para a súa execución ao SXBD. Ás veces implica converter as instrucións SQL a instrucións SQL do SXBD específico.
- ✓ A recuperación dos datos proporcionados polo SXBD, incluíndo a conversión de tipos de datos segundo o especificado pola aplicación.



Estrutura xeral de acceso a distintas bases de datos utilizando APIs standards e conectores

- ✓ A principal **vantaxe** é que a aplicación se **independiza do tipo de base de datos** utilizado para almacenar a información. Noutras palabras, non hai que escribir un programa para acceder a Oracle, outro para MySQL, etc... dado que a API Standard utiliza unha serie de clases e interfaces xenéricas que actúan sobre o conector, non sobre a base de datos. Tan só é necesario dispoñer dun conector específico para o tipo de base de datos co que se vai traballar.
- ✓ Como **inconveniente** sería a **penalización no rendemento** xa que a conexión entre a nosa aplicación e o motor de base de datos non é directa e necesita de intermediarios.

APIs estándar máis utilizadas:

- ✓ **JDBC (Java Database Connectivity)** – Java
- ✓ **ODBC (Open Database Connectivity)** – C/C++ e .NET
- ✓ **ADO.NET** – Entorno .NET (C#, VB.NET)
- ✓ **DB-API** (Python) – Python
- ✓ **PDO (PHP Data Objects)** – PHP

1.3 APIs standards para acceso a base de datos

JDBC (Java Database Connectivity) – Java

Descripción:

JDBC é a **API estándar de Java para acceder a bases de datos relacionais**. Forma parte do paquete `java.sql` e proporciona un conxunto de clases e interfaces que permiten conectar con calquera SXBD que teña un **driver JDBC**.

Elementos necesarios para usar JDBC:

- ✓ Base de datos (pode ser creada desde Java ou existente).
- ✓ SXBD como Oracle, MySQL, SQL Server, PostgreSQL, etc.
- ✓ Driver JDBC proporcionado polo fabricante do SXBD.

Funcionalidades principais:

- ✓ Conectar e desconectar dunha base de datos.
- ✓ Enviar instrucións SQL para execución.
- ✓ Recuperar datos e converter tipos de datos segundo as necesidades da aplicación.

Vantaxes:

- ✓ Independencia do motor de base de datos.
- ✓ Permite que o programador se concentre no desenvolvemento da aplicación sen preocuparse pola comunicación específica co SXBD.

Limitacións:

- ✓ A conexión non é directa, polo que pode existir unha pequena penalización no rendemento.

Exemplo de uso:

```
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/miBD",
"usuario", "contraseña");
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM clientes");
```

ODBC (Open Database Connectivity) – C/C++ e .NET

Descripción:

ODBC é un **estándar creado por Microsoft** para proporcionar **independencia entre aplicacións, bases de datos e sistemas operativos**.

Permite a **calquera aplicación** en C, C++ ou .NET conectarse a **calquera SXBD** que dispoña dun **driver ODBC**.

Funcionalidades principais:

- ✓ Traducir instrucións SQL xenéricas a comandos comprensibles polo SXBD.
- ✓ Recuperar datos da base de datos e converter tipos se é necesario.

Vantaxes:

- ✓ Independencia da linguaxe de programación e do sistema operativo.
- ✓ Amplamente soportada por moitos fabricantes de bases de datos.

Limitacións:

- ✓ Rendemento inferior comparado con drivers nativos.
- ✓ Necesita que tanto a aplicación como o SXBD sexan compatibles con ODBC.

Exemplo de uso (C++):

```
SQLConnect(hdbc, (SQLCHAR*)"miBD", SQL_NTS, (SQLCHAR*)"usuario", SQL_NTS,
(SQLCHAR*)"contraseña", SQL_NTS);
```

ADO.NET – Entorno .NET (C#, VB.NET)**Descrición:**

ADO.NET é a **API estándar en entornos .NET** para acceder a datos de **bases de datos relacionais e outras fontes de datos**. Substitúe a OLE DB/ODBC en aplicacións modernas.

Funcionalidades principais:

- ✓ Conexión e desconexión de bases de datos.
- ✓ Execución de consultas SQL e almacenadas.
- ✓ Traballo con datos en memoria mediante **DataSet** e **DataTable**.

Vantaxes:

- ✓ Integración completa con .NET.
- ✓ Seguro e eficiente.
- ✓ Permite traballar tanto con conexión directa como con datos en memoria.

Limitacións:

- ✓ Soporte limitado fóra do entorno .NET.

Exemplo de uso (C#):

```
using (SqlConnection conn = new SqlConnection("Server=localhost;Database=miBD;
User Id=usuario;Password=contraseña;"))
{
    conn.Open();
    SqlCommand cmd = new SqlCommand("SELECT * FROM clientes", conn);
    SqlDataReader reader = cmd.ExecuteReader();
}
```

DB-API – Python**Descrición:**

A **DB-API 2.0 (PEP 249)** é a **especificación estándar para acceder a bases de datos SQL en Python**. Cada SXBD require un driver Python que implemente esta API.

Funcionalidades principais:

- ✓ Conexión a bases de datos.
- ✓ Execución de consultas SQL e recuperación de resultados.

Vantaxes:

- ✓ Uniformidade na forma de acceder a distintas bases de datos.
- ✓ Drivers dispoñibles para MySQL, PostgreSQL, SQLite, Oracle, SQL Server, etc.

Limitacións:

- ✓ Cada driver pode ter pequenas diferenzas de implementación.

Exemplo de uso (PostgreSQL):

```
import psycopg2
conn = psycopg2.connect(dbname="miBD", user="usuario", password="contraseña",
host="localhost")
cur = conn.cursor()
cur.execute("SELECT * FROM clientes")
rows = cur.fetchall()
```

PDO (PHP Data Objects) – PHP**Descripción:**

PDO é a API estándar de PHP para acceder a bases de datos relacionais, proporcionando unha interface **unificada** para múltiples motores.

Funcionalidades principais:

- ✓ Conexión e desconexión de bases de datos.
- ✓ Execución de consultas SQL con preparación de sentenzas (prepared statements).
- ✓ Recuperación de resultados de forma consistente.

Vantaxes:

- ✓ Unifica o acceso a diferentes SXBD (MySQL, PostgreSQL, SQLite, Oracle, SQL Server).
- ✓ Mellora a seguridade evitando inxeccións SQL.

Limitacións:

- ✓ Só para PHP e bases de datos relacionais.

Exemplo de uso:

```
$pdo = new PDO('mysql:host=localhost;dbname=miBD', 'usuario', 'contraseña');
$stmt = $pdo->query("SELECT * FROM clientes");
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo $row['nombre'];
}
```

2. Base de Datos Independentes e Embebidas

Á hora de elixir unha base de datos para a nosa aplicación é importante coñecer as diferentes opcións que temos.

2.1 Base de datos independentes

A maioría dos SXBD máis populares e utilizados soportan a [arquitectura cliente/servidor](#).

- ✓ No servidor instálase e configúrase o SXBD como un servizo independente, onde se almacena a base de datos.
- ✓ Os programas dos clientes establecen unha conexión co servidor para emitir instrucións SQL e recibir os resultados.
- ✓ Os clientes só manexan peticións, procesamento e presentación dos datos; o servidor realiza a execución, xestión e control do acceso compartido aos datos.

Entre as características destes SXBD podemos citar :

- ✓ Son multiusuario e proporcionan unha alta taxa de concorrencia.

- ✓ Soportan transaccións.
- ✓ Teñen unha gran estabilidade e son moi robustos.
- ✓ Presentan unha gran escalabilidade.
- ✓ Manexan grandes volumes de datos.
- ✓ Subsistema de seguridade e autorización: garante protección fronte a accesos non autorizados, alteración ou destrución de datos, xestiona privilexios e realiza copias de seguridade.
- ✓ Ferramentas de auditoría: rexistro e monitorización de accesos, operacións e actividade do SXBD.

Resumo:

Os SXBD independentes proporcionan **unha solución robusta de persistencia de datos**, sendo adecuados cando os requisitos da aplicación inclúen rendemento, concorrencia, seguridade, escalabilidade, estabilidade e integridade referencial.

Exemplos de bases de datos independentes: **MSSQLSERVER, ORACLE, MySQL e PostgreSQL**, e **MariaDB** como alternativa moderna a MySQL, moi usada en contornos open source.

Evolución:

Estos sistemas tamén han evolucionado cara a **arquitecturas distribuídas e servizos na nube**.

Isto permite manter todas as características propias dos sistemas relacionais —transaccións, integridade referencial, concorrencia, seguridade— pero cun despregamento flexible que ofrece escalabilidade automática, alta dispoñibilidade e integración con contornos cloud como:

- **Azure SQL Database** (Microsoft SQL Server como servizo).
- **Amazon RDS** (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server).
- **Google Cloud SQL** (MySQL, PostgreSQL, SQL Server).

2.2 Bases de datos embebidas

Hai ocasións onde os requirimentos das aplicación son outros diferentes aos que aportan as base de datos independentes, como a **portabilidade**, utilización dunha **memoria mínima, rapidez de acceso**, e é aquí onde xurdiron pequenos motores denominados **bases de datos embebidas ou incrustadas**.

Características principais:

- ✓ **Arquitectura:** O SXBD non é un proceso independente, senón unha **biblioteca enlazada coa aplicación** mediante chamadas a funcións e subrutinas.
- ✓ **Rendemento:** Isto **reduce a latencia** e resulta en **alta velocidade** de acceso, xa que as chamadas a funcións son máis rápidas que a comunicación entre procesos (Cliente-Servidor).
- ✓ **Integración:** Ofrecen unha **mellor integración** co software debido ao seu pequeno tamaño e baixo consumo de recursos.
- ✓ **Dependencia:** Son **dependentes da aplicación**, instálanse xunto á aplicación e só esta pode acceder aos datos que xestionan.
- ✓ **Ciclo de Vida:** Iníciase ao executar a aplicación e péchanse ao pechar esta.
- ✓ **Recursos:** Caracterízanse polo seu **pequeno tamaño e baixo consumo de recursos**.
- ✓ **Acceso:** Accédese directamente ao ficheiro de datos sen necesidade de conexión cun servidor.

Resumo:

As bases de datos embebidas son ideais para **aplicacións que requiren portabilidade, rapidez de acceso, baixo consumo de recursos e uso exclusivo desde a aplicación que as incorpora**.

Exemplos: **SQLite** (a máis popular para aplicacións móbiles e de escritorio), Apache Derby, Hypersonic , Db2 de IBM e FireBird (con motor embebido con soporte SQL avanzado)

Característica	Independentes (Cliente/Servidor)	Embebidas (Incrustadas na aplicación)
Arquitectura	Servidor dedicado + clientes conectados vía rede	Biblioteca integrada directamente na aplicación
Uso principal	Aplicacións empresariais, web, sistemas multiusuario	Aplicacións móbiles, escritorio, probas de desenvolvemento
Escalabilidade	Alta (vertical e horizontal, incluso en cloud)	Limitada ao contexto da aplicación
Concorrenia	Multiusuario, alta taxa de concorrencia	Normalmente monousuario ou concorrencia reducida
Transaccións ACID	Soportadas plenamente	Soportadas, pero con limitacións segundo o motor
Seguridade	Subsistema avanzado de roles, privilexios, auditoría	Básica, restrinxida á propia aplicación
Volumen de datos	Grandes volumes, bases de datos corporativas	Volumes pequenos/medios, datos locais
Exemplos	Oracle, SQL Server, MySQL, MariaDB, PostgreSQL, IBM Db2	SQLite, H2, Apache Derby, Firebird
Evolución actual	Arquitecturas distribuídas e servizos cloud (Azure SQL, RDS)	Uso masivo en apps móbiles e software lixeiro

Comparativa: Bases de datos independientes vs embebidas

3. Acceso a base de datos relacionais dende Java: Conectividad JDBC

Java proporciona acceso a base de datos relacionais, dende a versión 1.1, empregando o conxunto de clases e interfaces da API JDBC.

Ano	Versión JDBC	Implementación JDK	Características destacadas
1997	JDBC 1.2	JDK 1.1	Clases básicas: <code>Connection</code> , <code>Statement</code> , <code>ResultSet</code>
1999	JDBC 2.1	JDK 1.2	<code>DataSource</code> , <code>PreparedStatement</code> , <code>ResultSet</code> scrollable
2001	JDBC 3.0	JDK 1.4	<code>Savepoint</code> , <code>batch updates</code> , <code>getGeneratedKeys()</code>
2006	JDBC 4.0	Java SE 6	Autocarga de drivers, soporte para SQL XML
2011	JDBC 4.1	Java SE 7	<code>try-with-resources</code> para peche automático de recursos
2014	JDBC 4.2	Java SE 8	Soporte para <code>java.time</code> , melloras en tipos SQL
2017	JDBC 4.3	Java SE 9+	Melloras en pooling, sharding e integración modular

Versións do API JDBC

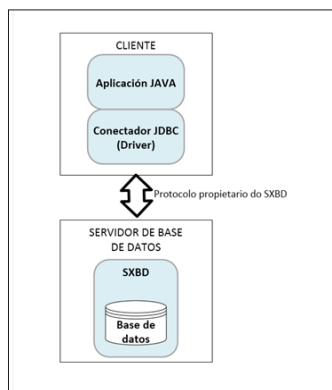
3.1 Modelos de acceso da API JDBC ás base de datos

A API JDBC soporta dous modelos distintos de acceso ás base de datos:

- Modelo de dúas capas.
- Modelo de tres capas.

Modelo de dúas capas

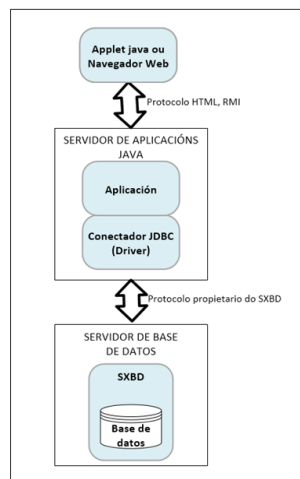
- ✓ Neste modelo a **aplicación JAVA** conéctase directamente co SXBD.
- ✓ Requírese que o **conectador JDBC se almacene xunto coa aplicación** (en el cliente) para que se poida comunicar co sistema xestor de bases de datos.
- ✓ As **sentenzas SQL** xeradas na aplicación cliente son enviadas á base de datos e os resultados da execución destas sentenzas son devoltos á aplicación.
- ✓ A aplicación cliente e a base de datos poden estar en máquinas diferentes.
- ✓ Esta é a arquitectura típica **cliente/servidor**, coa máquina do usuario que contén a aplicación como cliente, e a máquina que contén á base de datos e o SXBD como servidor.
- ✓ A comunicación entre o cliente e o servidor realízase a través da rede (Internet ou Intranet).



Modelo de dúas capas de conexión do conectador JDBC á base de datos

Modelo de tres capas

- ✓ Neste modelo, as sentenzas SQL son enviadas a unha capa intermedia de servizos que se encarga de envialas ao SXBD.
- ✓ O SXBD procesa as sentenzas e retorna os resultados á capa intermedia que se encarga de envialos á aplicación do usuario.



Modelo de tres capas de conexión do conectador JDBC á base de datos

Vantaxes:

- Control centralizado das operacións na base de datos a nivel intermedio.
- Non é necesario instalar os controladores JDBC en cada cliente.
- Uso: aplicacións web modernas e arquitecturas empresariais (J2EE / Jakarta EE).

3.2 Tipos de Conectores JDBC

Unha das decisións importantes no deseño dunha aplicación de bases de datos Java, é decidir o tipo de conector JDBC para a comunicación coa base de datos.

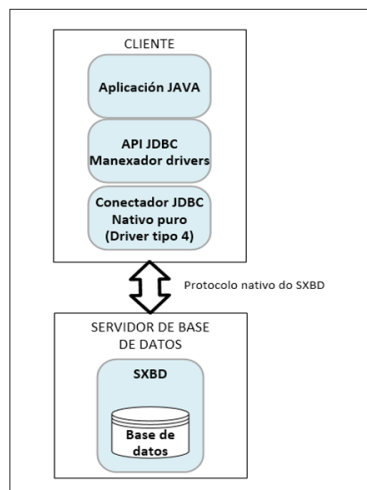
Hoxe, só o Tipo 4 é o estándar recomendado.

Os conectores JDBC clasifícanse en catro tipos ou niveis:

- Tipo 1: Ponte JDBC-ODBC
- Tipo 2: Conector API nativo/parte Java
- Tipo 3: Conector protocolo de rede/todo Java
- Tipo 4: Conector protocolo nativo/todo Java

Tipo 4: Conector de Protocolo Nativo

- ✓ **Descrición:** O estándar actual. Está escrito **integramente en Java** (100% *Pure Java*) e implementa o protocolo de comunicación específico do SXBD directamente.
- ✓ **Mecanismo:** O cliente Java **conéctase directamente ao SXBD** sen intermediarios.
- ✓ **Vantaxe Principal:**
 - **Portabilidade total (100% Java).**
 - **Rendemento óptimo** (o máis rápido, xa que non hai capas de tradución).
 - **Simplicidade:** Só se necesita o ficheiro .jar do *driver*.
- ✓ **Inconveniente:** **O driver é dependente do SXBD**, polo que se necesita un conector distinto para cada SXBD (necesitas o *driver* Tipo 4 de MySQL para MySQL, e o de Oracle para Oracle).



Funcionamento do conector tipo 4 JDBC protocolo nativo

Tipo 1: Ponte JDBC-ODBC (JDBC-ODBC bridge plus ODBC conector)

- ✓ **Descrición:** Foi o primeiro tipo de driver. Permitted o acceso a bases de datos non populares que só tiñan drivers ODBC (como Access ou dBase) mediante JDBC.
- ✓ **Mecanismo:** Traducía as chamadas JDBC en chamadas á API de ODBC do sistema operativo. Requiría a instalación e configuración dun driver ODBC no sistema cliente.

✓ **Vantaxe Principal:**

Foi o único xeito de acceder a certas bases de datos dende Java antes de que existisen drivers Tipo 4 para elas.

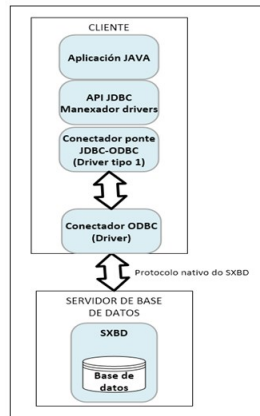
✓ **Inconveniente:**

Portabilidade nula: Dependía do código nativo (ODBC) do sistema operativo, o que impedía a portabilidade total de Java.

Rendemento lento: A conexión era máis lenta debido á dobre conversión (JDBC -> ODBC).

✓ **ESTADO CRÍTICO: FOI ELIMINADO COMPLETAMENTE de Java SE 8 e posteriores e o seu uso está desaconsellado.**

Os casos de uso actuais (como Access) resólvense con drivers Tipo 4 especializados (por exemplo, UCanAccess).



Funcionamento do conector tipo 1 ponte JDBC-ODBC.

Hai un conector xenérico ponte JDBC-ODBC incluído no JDK: `sun.jdbc.odbc.JdbcOdbcDriver`

Tipo 2: Un conector mixto Java - API Nativa

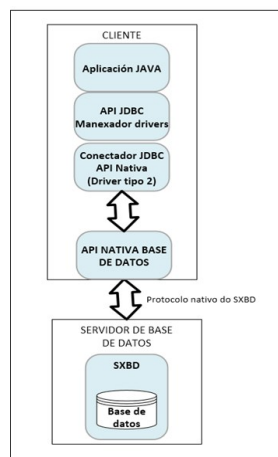
✓ **Descrición:** Este conector converte as chamadas JDBC en chamadas a métodos nativos do SXBD.✓ **Mecanismo:** Necesita unha librería específica proporcionada polo fabricante do SXBD (código nativo) para traducir as chamadas JDBC en código específico do SXBD. Do mesmo xeito que o Tipo 1, esixe a instalación e configuración da librería nativa en cada cliente.✓ **Vantaxe Principal:**

Mellor rendemento que o Tipo 1, xa que non pasa pola capa intermedia de ODBC.

✓ **Inconveniente:**

Portabilidade limitada, xa que o código nativo non se pode mover entre sistemas operativos.

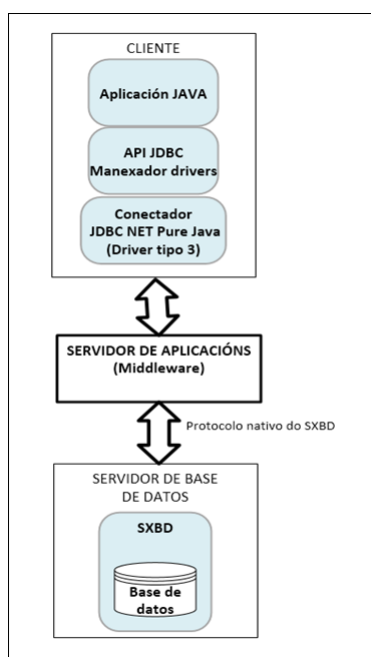
Non pode **utilizarse dentro de applets**.



Funcionamento do conector tipo 2 JDBC API Nativo

Tipo 3: Conector JDBC-Net puro Java

- ✓ **Descrición:** Está escrito **completamente en Java (100% Pure Java)** e comunícase cun servidor **intermedio (middleware)** utilizando un protocolo de rede xenérico.
- ✓ **Mecanismo:** O servidor intermedio encárgase de recibir as peticións e convertelas no protocolo **nativo do SXBD final**. O cliente non necesita saber nada sobre a base de datos final.
- ✓ **Vantaxe Principal:**
 - Portabilidade total no lado do cliente (100% Java).
 - Ideal para aplicacións con arquitectura de **tres capas**.
- ✓ **Inconveniente:**
 - O uso do **servidor intermedio engade complexidade**, latencia e un punto de fallo adicional ao sistema.



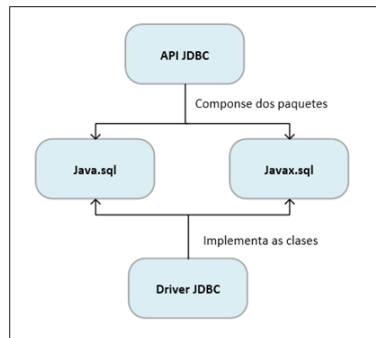
Funcionamento do conector tipo 3 JDBC NET puro nativo

Tipo	Descrición	Estado actual
Tipo 1: Ponte JDBC-ODBC	Traducía chamadas JDBC a ODBC. Dependía de código nativo e tiña baixo rendemento.	Eliminado desde Java 8. Non recomendado.
Tipo 2: Driver mixto Java + API nativa	Chamadas JDBC convertidas en chamadas a librerías nativas do SGBD. Mellor rendemento ca Tipo 1, pero pouca portabilidade.	Obsoleto. Pouco usado.
Tipo 3: Driver JDBC-Net (puro Java con servidor intermedio)	Cliente Java comunica cun servidor intermedio que traduce ao protocolo nativo do SGBD.	Case en desuso. Substituído por arquitecturas modernas.
Tipo 4: Driver de protocolo nativo (100% Java)	Comunícase directamente co SGBD usando o protocolo nativo. Portabilidade total e mellor rendemento.	Estándar actual e recomendado.

Comparativa dos tipos de conectadores JDBC

4. Paquetes da API JDBC

O API JDBC componse dun conxunto de interfaces Java que se atopan nos seguintes paquetes:



Nesta figura móstrase os paquetes que compoñen o API JDBC

■ **java.sql:** contén as clases e interfaces que compoñen a **funcionalidade básica** de JDBC:

- Conexión/desconexión á base de datos.
- Envío de peticións.
- Procesamento do resultado da execución de peticións.
- Lectura dos metadatos sobre a estrutura da base de datos.

Este paquete forma parte da linguaxe Java dende a liberación do JDK 1.1. Inclúese como parte da Plataforma Java 2, Standard Edition.

A partir de JDBC 4.0 (Java 6), as clases básicas adquiriron funcionalidades clave como a carga automática de drivers e, a partir de JDBC 4.1 (Java 7), o soporte para o peche automático de recursos (AutoCloseable).

■ **Javax.sql:** contén funcionalidade estendida, introducida no API JDBC 2.0.

As clases e interfaces neste paquete proporcionan unha nova funcionalidade, **como a agrupación de conexións** e as **transaccións distribuídas**, que non entran dentro do ámbito da API JDBC orixinal.

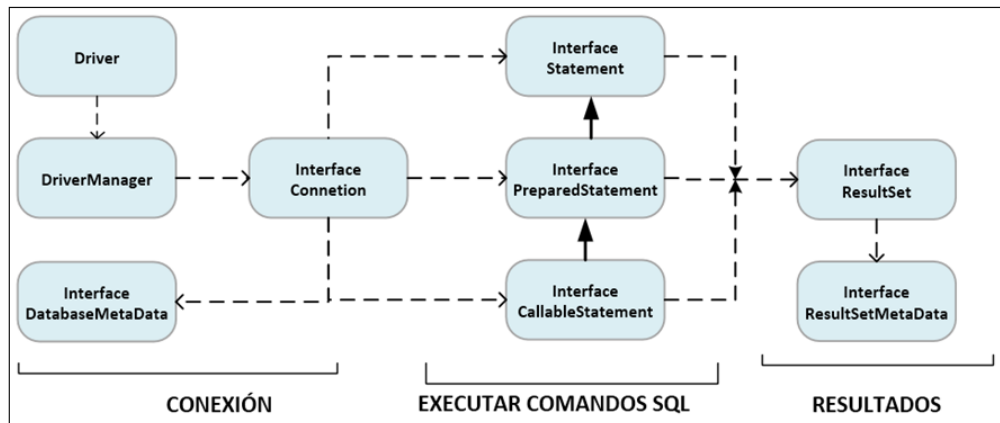
- ✓ **A través do agrupamento de conexións (pooling)**, permítese que unha conexión sexa utilizada e reutilizada, reducindo o número de conexións que é necesario crear e mellorando o rendemento.
- ✓ **O manexo de transaccións distribuídas** posibilita manexar nunha soa transacción, información provinte de distintas base de datos.

Paquete	Funcionalidades principais	Clases/Interfaces destacadas	Uso actual
java.sql	- Conexión/desconexión á BD - Envío de sentenzas SQL - Procesamento de resultados - Lectura de metadatos	DriverManager, Driver, Connection, DatabaseMetaData, Statement, PreparedStatement, CallableStatement, ResultSet, ResultSetMetaData	Base da API JDBC, incluído en Java SE desde JDK 1.1
javax.sql	- Funcionalidade estendida - Agrupamento de conexións (pooling) - Transaccións distribuídas - Integración con servidores de aplicacións	DataSource, ConnectionPoolDataSource, PooledConnection, XADataSource, RowSet	Usado en contornos empresariais (Jakarta EE, Spring), mellora rendemento e escalabilidade

Comparativa dos paquetes da API JDBC

4.1 Clases e interfaces do paquete java.sql

As clases e interfaces básicas da API JDBC son as seguintes:

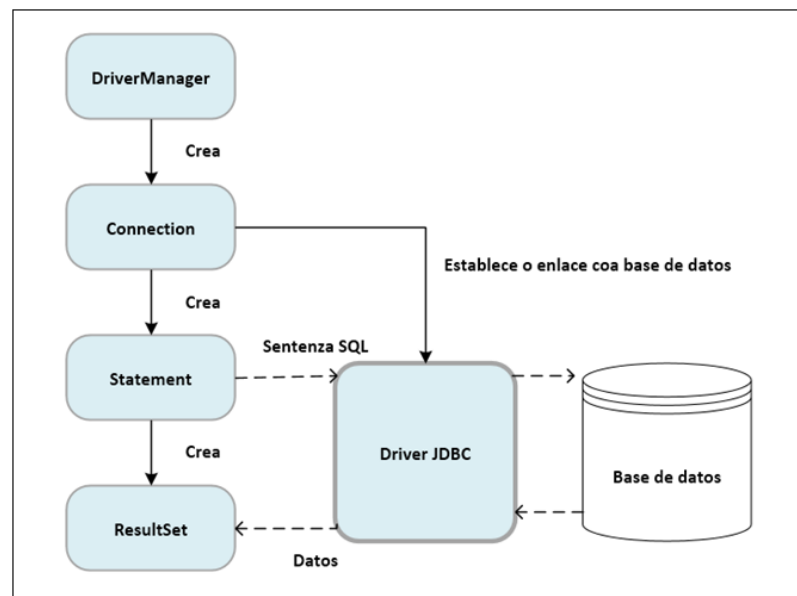


Nesta figura móstrase as clases e interfaces básicas que compoñen o paquete Java.sql

- A clase **DriverManager**
 - ✓ Esta clase **xestiona os conectadores da base de datos** e dá soporte á creación de novas conexións á base de datos. Encárgase de conectar a aplicación Java co conectador JDBC correcto.
 - ✓ Pódese utilizar DriverManager **para cargar varios conectadores en memoria á vez**. Cada conectador pode apuntar a unha base de datos distinta, ou todos eles poden apuntar á mesma.
- A interface **Driver**
 - ✓ Representa o **punto de contacto entre unha aplicación Java e o conectador** establece unha conexión coa base de datos.
- A interface **Connection**
 - ✓ Esta interface representa **unha sesión cunha base de datos específica**.
 - ✓ Permite crear obxectos **Statement, PreparedStatement y CallableStatement** para executar SQL.
- A interface **DatabaseMetaData**
 - ✓ Ademais é posible utilizar un obxecto Connection para **obter información sobre a estrutura da base de datos e sobre as capacidades do conectador JDBC** utilizando esta interface.
- As interfaces **Statement, PreparedStatement, e CallableStatement**
 - ✓ A interface Statement funciona como un contenedor para **executar instrucións SQL** nunha conexión dada. Existen dous subtipos importantes de Statement: PreparedStatement e CallableStatement.
 - ✓ A interface **PreparedStatement almacena unha instrución SQL precompilada**, de forma que pode ser executada varias veces. O seu uso **é a mellor práctica de seguridade** porque os datos se envían por separado do SQL, previndo a Inxección SQL.
 - ✓ A interface **CallableStatement executa procedementos almacenados e funcións SQL**.
- A interface **ResultSet**
 - ✓ A interface ResultSet **representa un conxunto de datos devoltos ao executar unha consulta SQL** de tipo SELECT, é dicir, é o resultado de executar unha instrución.
 - ✓ Dentro de cada fila é posible acceder aos valores das súas columnas en calquera orde.
 - ✓ Un ResultSet **mantén un cursor** que apunta a súa fila de datos actual, inicialmente o cursor sitúase antes da primeira fila.
 - ✓ O **método next** utilízase para mover o cursor á seguinte fila.
- A interface **ResultSetMetadata**
 - ✓ Obtén información **sobre o obxecto ResultSet**.

4.2 Pasos para acceder a unha base de datos utilizando API JDBC

- Instalar o conector.
- Cargar un conector de base de datos.
- Establecer conexións con bases de datos.
- Crear e executar instrucións SQL.
- Liberar os recursos e pechar a conexión.



Funcionamento básico dun programa que acceda a unha base de datos utilizando un conector JDBC

Instalación do conector

O conxunto de clases e interfaces do conector JDBC están implementados en arquivos con extensión `jar`, e para poder utilizalos teñen que estar cargados no proxecto Java.

- ✓ **Descargar o .jar correspondente** ao SGBD (ex: `mysql-connector-j`, `postgresql`, `Oracle JDBC (OJDBC)`, `Microsoft JDBC Driver for SQL Server`, etc..).
- ✓ **Engadilo ao classpath ou ao xestor de dependencias** (Maven/Gradle).
- ✓ **Mellor Práctica:** Usar sempre o **driver Tipo 4** (100% Java) para garantir a portabilidade.

Cargar o conector da base de datos

A clase **DriverManager** leva o **control dos conectadores JDBC cargados en memoria** e encárgase de **realizar as conexións** coa Base de Datos.

Cada **conector JDBC** é unha clase que implementa a interface **Driver** e deben ser rexistrados polo **DriverManager**.

Forma antiga (Carga Explícita do Conector)

Temos que *cargar en memoria os conectadores JDBC para que sexan rexistrados polo DriverManager*.

Hai varios xeitos de cargar e rexistrar o conector, pero o máis recomendado é o seguinte:

- ✓ Chamando ao **método estático `forName()`** da clase **Class**.

```
Class.forName ("jdbc.NomeClaseConector");
```

Este método carga en memoria a clase do conector e automaticamente é rexistrada polo **DriverManager**.

- ✓ A clase `Class` representa as instancias de todas as clases e interfaces que existen no programa actual. Non existe un construtor público para esta clase, os obxectos `Class` son instanciados directamente pola Máquina Virtual de Java.
- ✓ A función do método `Class.forName` é cargar unha clase de forma dinámica a partir do nome completo da clase que se lle pasa como parámetro.

Si o conector `jdbc.NomeClaseConector` está implementado correctamente, ao cargarse a súa clase lanzará automaticamente unha chamada ao método `DriverManager.registerDriver()`, e desta forma xa estará dispoñible na lista de conectores do `DriverManager` para que se poida realizar unha conexión con el.

O método `forName()` pode lanzar a excepción `ClassNotFoundException` cando se intenta cargar a clase do conector especificada e non se atopa.

Unha vez cargado o conector é posible realizar unha conexión coa base de datos.

Exemplo de cargar o conector JDBC de MySQL.

```
try {
    Class.forName ("com.mysql.cj.jdbc.Driver");
}
catch (ClassNotFoundException claseNoEncontrada) {
    System.out.println("Non se encontrou o conector");
}
```

Realizar a conexión coa base de datos

Unha vez que as clases dos conectores se cargaron na memoria e se rexistraron polo `DriverManager`, estas están dispoñibles para **establecer unha conexión cunha base de datos**.

Para establecer a conexión coa base de datos, úsase o método estático `getConnection()` da clase `DriverManager`.

```
public static Connection getConnection(url, usuario, contrasinal)
throws SQLException;
```

onde:

- ✓ url: é o identificador da Base de Datos.
- ✓ usuario: é o nome do usuario co que se abre a conexión (opcional).
- ✓ contrasinal: é o contrasinal do usuario (opcional) para acceder ao xestor da base de datos.

A conexión física coa base de datos represéntase en Java a través do interface: `Connection`.

```
Connection conexion = DriverManager.getConnection(url, usuario, contrasinal)
```

- ✓ Un obxecto de tipo `Connection` representa **unha sesión aberta cunha base de datos**.

Provén un contexto no que se poden emitir sentenzas SQL e obter resultados.

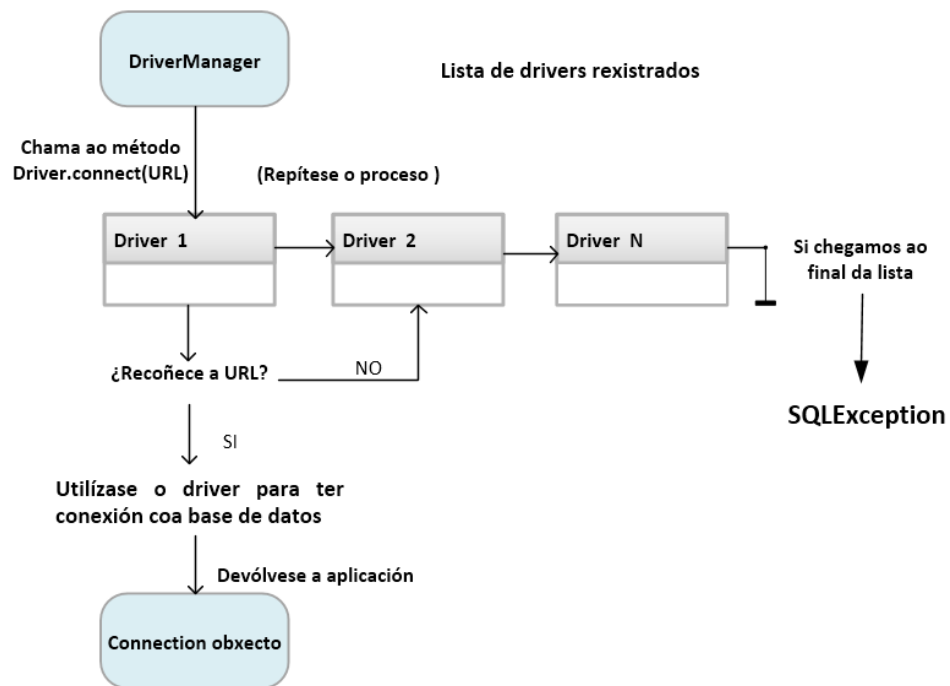
Unha aplicación pode ter varias conexións a unha mesma base de datos e/ou varias conexións a distintas bases de datos.

- ✓ Cando se realiza unha petición de conexión cunha chamada ao método `DriverManager.getConnection()`, a clase `DriverManager` vai testando cada un dos conectores na orde na que se rexistraron para comprobar si pode establecer a conexión.

O teste o realiza chamando ao método `Driver.connect` de cada conector, ao que pasa a URL de JDBC do método `DriverManager.getConnection()`.

O primeiro conector que recoñeza esta URL será o que realice a conexión e devolve un obxecto `Connection` á clase `DriverManager`, e que á súa vez o devolve á aplicación.

Si non localiza ningún conector que recoñeza a URL, lanzarase unha excepción específica de JDBC mediante a clase `SQLException`.



Nesta figura móstrase un esquema que representa todo o proceso de selección do conector JDBC adecuado

Exemplo de conexión á base de datos MySQL demodb.

```

Connection connexion;
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    connexion = DriverManager.getConnection("jdbc:mysql://localhost/demodb",
        "root", "abc123.");
}
catch(ClassNotFoundException claseNoEncontrada) {
    System.out.println("Non se atopou o conector");
}
catch(SQLException ex) {
    System.out.println("Erro ao realizar a conexión");
}

```

A conexión devolta polo método `DriverManager.getConnection` é unha conexión aberta que se pode utilizar para crear sentenzas JDBC.

Forma Moderna (Carga Automática do Conector)

A partir da especificación **JDBC 4.0 (lanzada con Java 6)**, a carga e rexistro do *driver* faise de forma automática, eliminando a necesidade do código explícito `Class.forName()`.

- ✓ **Contexto:** Esta funcionalidade é proporcionada polo mecanismo de **Servizo Provider Interface (SPI)** de Java, que mellorou significativamente a xestión dos *drivers*.
- ✓ **Mecanismo:** Ao iniciar a aplicación, o **DriverManager** **busca automaticamente os ficheiros de servizo dentro dos drivers JDBC** (ficheiros .jar no *classpath*). Estes ficheiros de servizo indican cal é a clase principal do *driver* (Driver), permitindo que o DriverManager a cargue e registre de forma transparente.
- ✓ **Resultado:** **Xa non é necesario** utilizar `Class.forName()` nin manexar a excepción `ClassNotFoundException` para este propósito. O único requisito é asegurar que o ficheiro .jar do *driver* estea correctamente no *classpath* da aplicación.
- ✓ **Conexión (Java 7+):** O obxecto **Connection** (xunto cos obxectos `Statement` e `ResultSet`) implementa a interface `AutoCloseable`. Polo tanto, a **conexión debe usarse SEMPRE** dentro do bloque **try-with-resources** para garantir que se peche de forma segura e automática, liberando recursos sen necesidade dun bloque `finally`.

Exemplo de Código Moderno:

```
// O driver de MySQL, PostgreSQL, Oracle, etc., cárgase automaticamente.
// Non se inclúe Class.forName().

try (Connection conn = DriverManager.getConnection(url, usuario, contrasinal)) {
    // A Connection úsase dentro dun try-with-resources
    // para garantir o peche automático (outra boa práctica moderna)
    // ...
} catch (SQLException e) {
    // Só se captura SQLException, que é lanzada polo DriverManager.
    e.printStackTrace();
}
```

Conclusión: Dende Java 6 en diante, o único paso necesario para que o DriverManager atope o *driver* é que o ficheiro .jar correspondente estea presente no *classpath*.

Identificación dunha base de datos mediante unha URL

A URL de JDBC proporciona a forma de identificar unha base de datos, de modo que o conector adecuado a reconeña e poida establecer unha conexión con ela. Do mesmo xeito que as URLs xerais serven para identificar, dun modo único, un recurso en Internet, neste caso unha URL de JDBC localizará unha base de datos determinada.

Cada provedor dun conector determina a URL que identifica ao seu conector particular.

A sintaxe estándar das URLs de JDBC ten tres partes, separadas por dous puntos (:)

```
jdbc:subprotocolo:subnome
```

Na URL, identifícanse o *protocolo*, *subprotocolo*, e a *base de datos específica*.

Por exemplo:

- ✓ protocolo sempre é **jdbc**.
- ✓ subprotocolo é o **nome do conector** ou o tipo de mecanismo de conexión á base de datos, que pode estar soportado por un ou máis conectores. Para que o DriverManager poida distinguir entre conectores, estes deben ter identificadores de subprotocolos distintos.
- ✓ subnome **identifica a base de datos**. O subnome pode variar, xa que pode identificar unha base de datos local, unha base de datos remota, un número de porto específico ou unha base de datos que requira identificador de usuario e contrasinal.

Si a base de datos se atopa nun servidor remoto é probable que sexa necesario ter máis información, por exemplo, pode ser necesario incluír como parte do subnome a dirección de rede do servidor. Neste caso, adóitase seguir o estándar de nomenclatura de URLs de Internet.

```
jdbc:subprotocolo://servidor:porto/subnome
```

Por exemplo, supoñamos que dbnet é un protocolo para conectarse a un servidor en rede.

```
jdbc:dbnet://miservidor:3560/mibase
```

Exemplos

SXBD	Clase Driver JDBC	URL de conexión típica	Porto por defecto
MySQL	com.mysql.cj.jdbc.Driver	jdbc:mysql://localhost:3306/nomeBD	3306
PostgreSQL	org.postgresql.Driver	jdbc:postgresql://localhost:5432/nomeBD	5432
Oracle	oracle.jdbc.OracleDriver	jdbc:oracle:thin:@localhost:1521:nomeBD	1521
SQL Server	com.microsoft.sqlserver.jdbc.SQLServerDriver	jdbc:sqlserver://localhost:1433;databaseName=nomeBD	1433
SQLite	org.sqlite.JDBC	jdbc:sqlite:ruta/nomeBD.db	(sen porto, é ficheiro local)

4.3 O interface Connection

Neste apartado imos comentar de forma breve algúns dos distintos métodos que ofrece o interface `Connection` a modo de referencia rápida. A medida que vaiamos avanzando na unidade didáctica iremos retomando algúns deles para profundar máis na súa explicación.

■ Crear e executar sentenzas SQL ou procedementos almacenados

■ **Statement** `createStatement()`

Crea unha sentenza SQL, representada mediante un obxecto `Statement`, para poder enviar sentenzas á base de datos.

■ **PreparedStatement** `prepareStatement(String sql)`

Crea un obxecto `PreparedStatement` que vai permitir enviar e executar sentenzas SQL parametrizadas á base de datos correspondente.

■ **CallableStatement** `prepareCall(String sql)`

Crea un obxecto `CallableStatement` que vai permitir chamar e executar procedementos almacenados ou funcións da base de datos á que estamos conectados.

■ Transaccións

■ `void setAutoCommit(boolean autoCommit)`

Establece o modo `autocommit` da conexión para tratar as transaccións. Úsase para controlar explicitamente as confirmacións das modificacións realizadas ou desfacer os cambios.

■ `boolean getAutoCommit()`

Comproba si a conexión se atopa en estado de `autocommit`.

■ `void commit()`

Leva a cabo todos os cambios realizados dentro dunha transacción e libera todos os bloqueos correspondentes.

■ `void rollback()`

Desfai as modificacións realizadas nunha transacción e libera todos os bloqueos asociados.

■ `int getTransactionIsolation()`

Devolve o grado de illamento que se estableceu para a conexión e que se utilizará á hora de realizar transaccións.

■ `void setTransactionIsolation(int nivel)`

Establece o nivel de illamento dunha conexión á hora de executar e tratar transaccións.

■ Pechar ou ver o estado da conexión

■ `void close()`

Pecha unha conexión, liberando todos os recursos asociados á mesma.

■ `boolean isClosed()`

Devolve verdadeiro si a conexión está pechada.

■ Información da base de datos conectada (Metadatos)

■ `DatabaseMetaData getMetaData()`

Devolve un obxecto `DatabaseMetaData` que contén información detallada sobre a base de datos á que nos atopamos conectados

■ Ver ou establecer o modo de lectura ou escritura da conexión

- void **setReadOnly**(boolean soloLectura)
Establece si unha conexión posúe o modo de só lectura ou non, dependendo do argumento de tipo booleano.
- boolean **isReadOnly**()
Devolve `true` si a conexión é de só lectura.

4.3.1 Crear e executar instrucións SQL

Unha vez que se estableceu a conexión á base de datos, emprégase o obxecto `Connection` para enviar sentenzas SQL ao xestor.

As sentenzas SQL pódense enviar ao xestor utilizando tres interfaces diferentes:

- **Statement**: creación e execución de sentenzas SQL de forma estática.
- **PreparedStatement**: creación e execución de sentenzas SQL de forma dinámica.
- **CallableStatement**: execución de procedementos almacenados no xestor da base de datos.

Creando un obxecto Statement

Unha vez que se estableceu unha conexión a unha base de datos determinada, esta conexión pódese utilizar para enviar sentenzas SQL á base de datos.

Un obxecto `Statement` créase co método `createStatement()` da clase `Connection`, como se pode observar no fragmento de código seguinte:

```
Connection conexion=DriverManager.getConnection(url,"pepa","abc123.");
Statement sentenza=conexion.createStatement();
```

Execución de obxectos Statement

O interface `Statement` ofrece os métodos `executeQuery()` e `executeUpdate()` para executar sentenzas SQL.

O uso de cada un deles vén determinado polo tipo de sentenza SQL que conteña e polo resultado que devolva.

- **Sentenzas SELECT**
 - Úsase o método: `executeQuery(String sql);`
 - Devolve unha instancia de `java.sql.ResultSet`
- **Sentenzas INSERT, UPDATE e DELETE**
 - Úsase o método: `executeUpdate(String sql);`
 - Devolve un enteiro que indica o número de filas que foron afectadas .
- **Sentenzas CREATE TABLE e DROP TABLE**
 - Úsase o método: `executeUpdate(String sql);`
 - Devolve un enteiro que sempre vale 0, xa que non operan con filas.

Exemplo de executar unha sentenza SELECT

```
ResultSet rs=sentenza.executeQuery("SELECT nome, apelido1, apelido2
FROM empregado");
```

Exemplo de executar unha sentenza INSERT

```
int filas=sentenza.executeUpdate("INSERT INTO departamento
VALUES (1,'Contabilidade','2001-2-2',11111111)");
```

Exemplo de executar unha sentenza UPDATE

```
int filas=sentenza.executeUpdate("UPDATE SET comision=comision*(1.20)
FROM empleado WHERE codigo = 15");
```

Exemplo de executar unha sentenza DELETE

```
int filas=sentenza.executeUpdate("DELETE FROM alumno WHERE id = 15");
```

Exemplo de executar unha sentenza CREATE TABLE

```
int filas=sentenza.executeUpdate("CREATE TABLE categoria( id int primary
key, descripcion varchar(15) not null)");
```

Método xeral execute() (serve para todo tipo de SQL)

Este método permite executar **calquera** sentenza SQL, pero é máis complexo.

Método:

- boolean tenResultado = sentenza.execute(String sql);
- **Devolve:**
 - true → hai un **ResultSet**
 - false → o resultado é un **número de filas afectadas** (GET mediante getUpdateCount())

Exemplo completo usando execute():

```
boolean tenResultado = sentenza.execute("SELECT * FROM empleado");

if (tenResultado) {
    ResultSet rs = sentenza.getResultSet();
    // procesar o resultado
} else {
    int filas = sentenza.getUpdateCount();
    // número de filas afectadas
}
```

É útil cando:

- Non sabemos que tipo de SQL chega (por exemplo, nun editor SQL).
- Hai procedementos almacenados que devolven varios resultados.

Pegar as conexións e liberar os recursos

■ Pechado tradicional (sen try-with-resources)

É recomendable que se pechen os obxectos `Connection` e `Statement` creados cando xa non se necesiten.

Cando nunha aplicación en Java, se están usando recursos externos, como é o caso do acceso a una base de datos mediante a API JDBC, o recolector de lixo de Java (garbage collector) non ten xeito de coñecer cal é o estado deses recursos, e polo tanto, non é capaz de liberalos no caso de que xa non sexan útiles.

Pode ocorrer que en memoria se queden almacenadas grandes cantidades de recursos relacionados coa aplicación de bases de datos que se está executando, para liberar estes recursos **recoméndase pechar de xeito explícito os obxectos `Connection` e `Statement`**.

As interfaces `Connection` e `Statement` teñen un método `close()` que permite pechalos de xeito explícito e así liberar os recursos que están en uso, tanto na aplicación Java como no servidor de bases de datos.

Si se creou un obxecto `ResultSet`, este tamén se debería pechar utilizando para iso o seu método `close()`, igual que para o obxecto `Connection` ou `Statement`. Aínda que o obxecto `ResultSet` se pecha automaticamente ao pechar o obxecto `Statement` que o creou, é mellor acostumar a pechalo.

As sentenzas péchanse mediante o método:

```
public void close() throws java.sql.SQLException;
```


■ Pechado automático con try-with-resources (recomendado)

A partir de Java 7, pódese usar o mecanismo *try-with-resources*, no que **todos os recursos que implementan `AutoCloseable`** (`Connection`, `Statement` e `ResultSet` o fan) se pechan automaticamente ao saír do bloque `try`.

```
String sql = "SELECT * FROM empregado";
try (
    Connection con = DriverManager.getConnection(url, usuario,
                                                contrasinal);
    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery(sql)
) {
    // procesar datos...
} catch (SQLException e) {
    e.printStackTrace();
}
```

Vantaxes de try-with-resources:

- Pecha automaticamente todos os recursos.
- Código máis curto e limpo.
- Menos risco de fugas de memoria ou conexións.
- Cada recurso pechase na orde inversa á apertura (correcto segundo JDBC).

Xestión de erros

Cada vez que realicemos algunha acción na base de datos, como abrir a conexión, pechala, executar unha sentenza SQL, etc., pódese lanzar a excepción `SQLException` si acontece algún erro, o que deberemos xestionar.

A clase `java.sql.SQLException` herda da clase `java.lang.Exception`, e particulariza o tratamento das excepcións producidas no acceso ás bases de datos relacionais.

A clase `SQLException` posúe algunhas particularidades que a diferenzan do resto das excepcións. Estas particularidades son:

A excepción `SQLException` *pode ter encadeados varios obxectos `SQLException`*, é dicir, si se producen catro excepcións nun acceso a unha base de datos, o obxecto `SQLException` que se instancia ten unha lista coas catro excepcións.

Posúe métodos que non ten a clase `java.lang.Exception`. Os principais métodos da clase `SQLException` son os seguintes:

- **`getErrorCode()`**: devólvenos o código de erro. Este código será específico de cada fabricante de SGBD.
- **`getSQLState()`**: devólvenos o estado SQL que se corresponde co estándar X/OPEN SQL.
- **`getNextException()`**: obtemos a seguinte excepción producida, é dicir, desprázase á seguinte `SQLException` dentro da lista das excepcións producidas.
- **`getMessage()`**: devólvenos un obxecto `String` que describe a excepción que se produciu. Este método herda da clase `Exception`.

O seguinte exemplo mostra información das excepcións que puideron acontecer.

```
public static void mostraSQLException(SQLException ex) {
    System.out.println("Se han dado excepciones SQLException\n");
    //Poden existir varias SQLException encadenadas
    while (ex != null) {
        System.out.println("Mensaje : "+ex.getMessage()+"\n");
        System.out.println("estado SQL : "+ex.getSQLState()+"\n");
        System.out.println("Código do erro : "+ex.getErrorCode()+"\n");
        ex = ex.getNextException();
        System.out.println("\n");
    }
}
```


}

Exemplo dunha aplicación básica de acceso a base de datos.

Nela vaise a acceder á base de datos MySQL, chamada empresa, para visualizar e inserir un novo departamento.

```
import java.sql.*;
public class ExemploAccesoBD {
    public static void main(String[] args) {
        Connection conexion = null;
        try {
            // Cargar o conector
            Class.forName("com.mysql.jdbc.conector");
            // Obtense unha conexión coa base de datos.
            // Neste caso conectámonos á base de datos empresa
            conexion = DriverManager.getConnection
                ("jdbc:mysql://localhost/empresa", "root", "abc123.");
            // Créase un obxecto Statement, para realizar a consulta
            Statement sentenxa = conexion.createStatement();
            // Realízase a inserción do novo departamento
            int filas=sentenxa.executeUpdate("INSERT INTO departamento
                VALUES(8,'Contabilidade','2001-2-2',11111111) ");
            sentenxa.close();//péchase o Statement
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        } catch (ClassNotFoundException e) {
            System.out.println(e.getMessage());
        } finally {
            // Péchase a conexión coa base de datos.
            if (conexion != null) {
                conexion.close();
            }
        }
    }
}
```

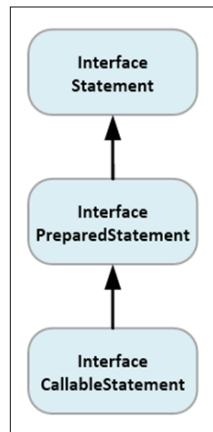
5. Execución de sentenzas de consulta e modificación de datos e procedementos almacenados.

Unha vez establecida a conexión coa base de datos, poderemos enviar sentenzas SQL para realizar operacións contra ela. Existen tres tipos de obxectos que representan unha sentenza SQL: Statement, PreparedStatement e CallableStatement do paquete java.sql.

PreparedStatement herda da interface Statement e CallableStatement da interface PreparedStatement. Na seguinte figura móstrase a xerarquía destas tres interfaces:

Cada unha de estas interfaces está especializada para enviar un tipo específico de sentenza SQL:

- **Statement** é utilizada para crear un obxecto que executa unha sentenza SQL simple e sen parámetros.
- **PreparedStatement** é utilizada para crear un obxecto que executa sentenzas SQL precompiladas, normalmente, con parámetros de entrada.
- **CallableStatement** é utilizada para crear un obxecto que executa unha chamada a un procedemento almacenado ou a unha función.



Nesta imaxe mostrase a herdanza da interface Statement, PreparedStatement e CallableStatement

A interface `Statement` ofrece métodos básicos para executar sentenzas e devolver os resultados. A interface `PreparedStatement` engade métodos para executar sentenzas que posúen parámetros de entrada e a interface `CallableStatement` engade métodos para tratar cos parámetros de saída.

5.1 Execución de sentenzas SQL simples: A interface Statement

Creando un obxecto Statement

Antes de poder executar unha sentenza SQL, é necesario obter un obxecto de tipo `Statement`. Unha vez creado o devandito obxecto, poderá ser utilizado para executar calquera operación contra a base de datos.

Sintaxe:

```
Statement createStatement() throws SQLException
```

Un obxecto `Statement` créase co método `createStatement()` da clase `Connection`, como se pode observar no fragmento de código seguinte:

```
Connection conexion=DriverManager.getConnection(url,"luis","abc123.");
Statement sentenza=conexion.createStatement();
```

Métodos da interface Statement

Imos ver unha referencia rápida dalgúns dos métodos que nos presenta esta interface:

- `void addBatch(String sql) throws SQLException`
Engade a sentenza SQL a un lote de sentenzas.
- `void cancel() throws SQLException`
Cancela o obxecto `Statement`, abortando a sentenza SQL correspondente.
- `void clearBatch() throws SQLException`
Elimina todas as sentenzas do lote.
- `void clearWarnings() throws SQLException`
Elimina todos os avisos ofrecidos polo obxecto `Statement`.
- `void close() throws SQLException`
Pecha o obxecto `Statement`, liberando todos os recursos asociados.

- boolean **execute**(String sql) throws SQLException
Executa a sentença SQL.
- int[] **executeBatch**() throws SQLException
Envía un conxunto de sentenzas SQL á base de datos para que se executen como un lote. Devolve un array co número de filas afectadas de cada sentença SQL.
- ResultSet **executeQuery**(String sql) throws SQLException
Executa unha sentença SQL que devolve un conxunto de resultados, representado polo obxecto ResultSet.
- int **executeUpdate**(String sql) throws SQLException
Executa a sentença SQL. A sentença SQL debe ser unha sentença DML como INSERT, UPDATE e DELETE ou unha sentença DDL como CREATE TABLE, DROP TABLE e ALTER TABLE.
- Connection **getConnection**() throws SQLException
Devolve o obxecto Connection do que se creou o obxecto Statement.
- int **getFetchDirection**() throws SQLException
Devolve a dirección que se utiliza para ir recuperando rexistros. Devolve unha constante definida no interface ResultSet, que pode ser FETCH_FORWARD (cara a adiante), FETCH_REVERSE (cara a atrás) ou FETCH_UNKNOWN (dirección descoñecida).
- int **getFetchSize**() throws SQLException
Devolve o número de rexistros que se recuperan da base de datos cada vez que se necesitan máis rexistros.
- int **getMaxFieldSize**() throws SQLException
Devolve o número máximo de bytes que se permite para un campo.
- int **getMaxRows**() throws SQLException
Devolve o número máximo de rexistros que un obxecto ResultSet pode conter como resultado da execución dun obxecto Statement.
- boolean **getMoreResults**() throws SQLException
Desprázase ao seguinte resultado obtido a partir da execución dun obxecto Statement.
- int **getQueryTimeout**() throws SQLException
Devolve o número de segundos *que o conector vai esperar para que se execute un obxecto Statement*.
- ResultSet **getResultSet**() throws SQLException
Devolve o resultado actual, en forma dun obxecto ResultSet.
- int **getResultSetConcurrency**() throws SQLException
Devolve o tipo de concorrencia aplicada aos obxectos ResultSet, que se obteñan a partir do obxecto Statement actual. O valor devolto correspóndese cunha serie de constantes definidas no interface ResultSet.
- int **getResultSetType**() throws SQLException
Devolve o tipo de ResultSet que se vai utilizar para un obxecto Statement.
- int **getUpdateCount**() throws SQLException
Devolve o resultado actual como un número de actualizacións realizadas (filas afectadas). Si o resultado é un ResultSet ou non hai máis resultados, devolve 1.
- SQLWarning **getWarnings**() throws SQLException
Devolve o primeiro aviso (obxecto SQLWarning) xerado polas chamadas ao obxecto Statement.

- void **setCursorName()** throws SQLException
Define un cursor SQL que vai ser utilizado nas distintas chamadas ao método execute() do obxecto Statement.
- void **setFetchDirection**(int dirección) throws SQLException
*Indica ao conector JDBC, en que dirección debe devolver os rexistros *que se vaian necesitando*.*
- void **setFetchSize**(int rexistros) throws SQLException
Indica ao conector JDBC, o número de rexistros que se deben obter da base de datos, cada vez que se vaian necesitando.
- void **setMaxFieldSize**(int máximo) throws SQLException
Establece o máximo número de bytes que pode ter un campo.
- void **setMaxRows**(int máximo) throws SQLException
Establece o número máximo de rexistros que pode conter un obxecto ResultSet.
- void **setQueryTimeout**(int segundo) throws SQLException
Establece o número de segundos que o conector JDBC esperará para que se execute un obxecto Statement.

Execución de sentenzas mediante obxectos Statement

A finalidade dun obxecto `Statement` é executar unha instrución SQL. Ao executar a sentenza SQL pódese ou non devolver resultados.

A interface `Statement` dispón dos seguinte métodos:

- **executeQuery(String sql)**
Para executar sentenzas `SELECT` que recuperen datos dun único obxecto `ResultSet`.
 - **executeUpdate(String sql)**
Para realizar actualizacións que non devolvan un `ResultSet`.
 - Sentenzas DML como `INSERT`, `UPDATE` e `DELETE`
 - Sentenzas DDL como `CREATE TABLE`, `DROP TABLE` e `ALTER TABLE`.
- O valor que devolve `executeUpdate()` é un enteiro
- Indica o número de filas que se viron afectadas.
 - As sentenzas que non operan en filas, como `CREATE TABLE` ou `DROP TABLE`, devolven o valor cero.

Execución de sentenzas de Manipulación de datos

As sentenzas de manipulación de datos (DML) son as utilizadas para agregar, actualizar ou eliminar datos dunha base de datos.

As sentenzas son as seguintes:

- **INSERT:** Engade novas filas de datos á base de datos.
- **DELETE:** Suprime filas de datos da base de datos.
- **UPDATE:** Modifica datos.

Para executar as sentenzas DML utilízase o método **executeUpdate()**.

Como realizar unha inserción

A sintaxe da sentenza `INSERT` para engadir unha fila nunha táboa é:

```
INSERT INTO <nome táboa> [(<lista de columnas>)]
VALUES (<lista de valores>)
```

A sintaxe da sentenza INSERT para engadir varias filas nunha táboa é:

```
INSERT INTO <nome táboa> [(<lista-de columnas>)] <sentenza SELECT>
```

Exemplo:

```
// define a sentenza de inserción e execútaa
String SQL = "INSERT INTO alumnos (alu_id, alu_nome, alu_apellido)
              VALUES (101, 'Manuel', 'Santos')";
Statement stmtInsercion = conexion.createStatement();
stmtInsercion.executeUpdate(SQL); //tb vale o método execute()
// Pecha o Statement
stmtInsercion.close();
```

Como realizar unha actualización

A sintaxe da sentenza UPDATE para modificar datos nunha táboa é:

```
UPDATE <nome táboa> SET { <columna> = <expresión> [, ...]
| {( <lista-de columnas> ) | *} = ( <lista-de expresións> ) }
[WHERE <condición>]
```

Exemplo:

```
//define a sentenza Update
String SQL = "UPDATE alumnos SET alu_apellido = 'Sergio' WHERE alu_id = 18";
//crea o obxecto statement
Statement stmtActualizacion = conexion.createStatement();
//executa a sentenza de actualización
int filas=stmtActualizacion.executeUpdate(SQL); //tb vale o método execute()
// Pecha o obxecto Statement
stmtActualizacion.close();
```

Como realizar unha eliminación

A sintaxe da sentenza DELETE para borrar datos nunha táboa é:

```
DELETE FROM <nome táboa> [WHERE <condición>]
```

Exemplo:

```
// Define a sentenza de eliminación e execútaa
String SQL = "DELETE FROM alumnos WHERE alu_id = 101";
Statement stmtEliminacion = conexion.createStatement();
int filas=stmtEliminacion.executeUpdate(SQL); //tb vale o método execute()
// Pecha o Statement
stmtEliminacion.close();
```

Execución de consultas

A sentencia SELECT permite consultar os datos da base de datos.

Sintaxe:

```
SELECT [ALL | DISTINCT] <lista-de seleccións>
FROM <nome táboa> [alias de táboa] [,...]
[WHERE <condición>]
[GROUP BY <lista-de columnas>]
[HAVING <condición>] ]
[ORDER BY <nome de columna> [ASC | DESC] [,...] ]
```

Exemplo:

```
String SQL = "SELECT * FROM alumnos";
// executa a consulta
Statement stmtConsulta = conexion.createStatement();
ResultSet rs = stmtConsulta.executeQuery(SQL);

// Mostra os datos
while(rs.next()){
    System.out.println( "ID:"+rs.getInt("alu_id")+" " +"Nome:"+
        rs.getString("alu_nome")+" " +"Apellido:"+rs.getString("alu_apellido"));
}

// Pecha o obxecto Statement
stmtConsulta.close();
```

Percorrer o resultado dunha consulta

O obxecto `ResultSet` vaise a crear a partir da execución dun obxecto `Statement`. Por defecto, créase un obxecto `ResultSet` con movemento unicamente cara adiante (de tipo `FORWARD_ONLY`) e só de lectura (de tipo `CONCUR_READ_ONLY`).

No obxecto `ResultSet`, atópanse os resultados da execución dunha sentenza SQL, polo tanto, contén as filas que satisfán as condicións dunha sentenza SQL. Ofrece o acceso aos datos das filas, a través dunha serie de métodos `getXXX`. Estes métodos permiten acceder ás columnas da fila actual.

Implementa métodos para:

- **Acceder ás filas** que compoñen o resultado. O método para acceder á fila seguinte do resultado é:

```
public boolean next() throws SQLException;
```

- **Acceder ao valor de cada columna** da fila seleccionada.

O método para acceder ao valor dunha columna dependerá do tipo de dato, o valor do índice é 1 para a primeira columna. A sintaxe xeral do método `getXXX` é a seguinte, onde `XXX` é o tipo de datos:

```
public xxxx getXXX(int columna) throws SQLException;
```

O método `next()` do interface `ResultSet`

Utilízase para desprazarnos ao rexistro seguinte dentro dun `ResultSet`. O método `next()` devolve un valor `true` si o rexistro seguinte existe e `false` si chegamos ao final do obxecto `ResultSet`, é dicir, non hai máis rexistros.

Para percorrer un `ResultSet` completo cara a adiante, farémolo a través dun bucle `while`, no que se lanza o método `next()` sobre o obxecto `ResultSet`. A execución deste bucle finalizará cando alcancemos o final do conxunto de rexistros.

Á vez que imos percorrendo o obxecto `ResultSet`, obteremos os valores dos campos do rexistro actual, mediante os métodos `getXXX()` correspondentes. A continuación, móstrase un exemplo:

```
ResultSet rs=sentencia.executeQuery("SELECT nome, apellido1, apellido2
                                     FROM empleado");

while (rs.next()){
    rs.getXXX(1);
    .....
    rs.getXXX(n);
}
```

Os métodos `getXXX()`

Cando se lanza un método `getXXX()` determinado sobre un obxecto `ResultSet`, para obter o valor dun campo, o conector JDBC converte o dato que se quere recuperar ao tipo Java especificado e devolve un valor Java axeitado.

Por exemplo, si utilizamos o método `getString()` e o tipo do dato na base de datos é `VARCHAR`, o conector JDBC converterá o dato `VARCHAR` a un obxecto `String` de Java, polo tanto, o valor de retorno de `getString()` será un obxecto da clase `String`.

Esta conversión de tipos pódese realizar grazas á clase `java.sql.Types`. Nesta clase defínense o que se denominan *tipos de datos JDBC*, que se corresponden cos tipos de datos SQL estándar.

Temos dúas alternativas para acceder ás columnas do `ResultSet`:

- **Polo nome do campo:** `rs.getString("nome");`
- **Pola posición da columna** dentro do `ResultSet`: `rs.getString(1);`

Si nestes métodos se utiliza o índice da columna, débese considerar que os índices empezan a partir de 1, e non en 0 (cero), como nos vectores e algunhas outras estruturas de datos de Java.

O mapeo ou conversión que se realiza entre tipos JDBC e clases Java, podémolo observar na táboa que aparece a continuación:

		SMALLINT	INTEGER	BIGINT	REAL	FLOAT	DOUBLE	DECIMAL	NUMERIC	BIT	CHAR	VARCHAR	LONGVARCHAR	BINARY	VARBINARY	LONGVARBINARY	DATE	TIME	TIMESTAMP
<code>getBytes</code>		X	x	x	x	x	x	x	x	x	x	x							
<code>getShort</code>		x	X	x	x	x	x	x	x	x	x	x							
<code>getInt</code>		x	x	X	x	x	x	x	x	x	x	x							
<code>getLong</code>		x	x	x	X	x	x	x	x	x	x	x							
<code>getFloat</code>		x	x	x	x	X	x	x	x	x	x	x							
<code>getDouble</code>		x	x	x	x	x	X	x	x	x	x	x							
<code>getBigDecimal</code>		x	x	x	x	x	x	X	x	x	x	x							
<code>getBoolean</code>		x	x	x	x	x	x	x	x	X	x	x							
<code>getString</code>		x	x	x	x	x	x	x	x	x	X	x					x	x	x
<code>getBytes</code>														X	X	x			
<code>getDate</code>											x	x	x				X		x
<code>getTime</code>											x	x	x					X	x
<code>getTimestamp</code>											x	x	x				x		X
<code>getAsciiStream</code>											x	x	X	x	x	x			
<code>getUnicodeStream</code>											x	x	X	x	x	x			
<code>getBinaryStream</code>														x	x	X			
<code>getObject</code>		x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

Uso dos métodos `getXXX` do obxecto `ResultSet` que reciben os tipos de datos SQL
 Unha x significa que o método `getXXX` pode ser usado para recibir o tipo de dato SQL
 Unha X significa que o método `getXXX` é o recomendado para recibir o tipo de datos SQL

Para determinar si un valor devolto foi nulo, é dicir un JDBC NULL, primeiro débese ler a columna e logo lanzar o método `wasNull()` do interface `ResultSet`, que devolverá true ou false segundo a situación. O método `wasNull()` devolverá true nos seguintes casos:

- Obtense un valor NULL de Java para métodos `getXXX` que devolvan obxectos Java (métodos tales como: `getString()`, `getBigDecimal()`, `getBytes()`, `getDate()`, `getTime()`, `getTimestamp()`, `getAsciiStream()`, `getUnicodeStream()`, `getBinaryStream()`, `getObject()`).
- Obtense un cero devolto polo métodos: `getByte()`, `getShort()`, `getInt()`, `getLong()`, `getFloat()` e `getDouble()`.
- O método `getBoolean()` devólvenos false.

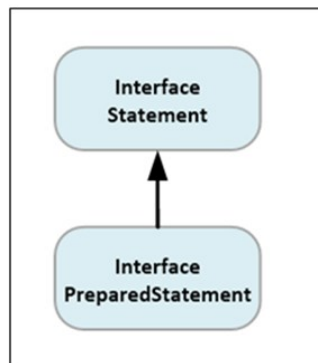
Pechar os obxectos

Os obxectos `Statement` pecharaos automaticamente o colleiteiro de lixo (garbage collector) de Java. Non obstante, *é recomendable pechar explicitamente os obxectos `Statement`*, unha vez que xa non se vaian necesitar máis. Isto libera inmediatamente, os recursos do SXBD e axuda a evitar problemas potenciais de memoria.

Para pechar unha sentenza, deberase utilizar o método `close()` do interface `Statement`.

5.2 Execución de sentenzas precompiladas: A interface `PreparedStatement`

Esta interface, do mesmo xeito que a interface `Statement`, permítenos executar sentenzas SQL sobre unha conexión establecida cunha base de datos. Pero neste caso, imos executar sentenzas SQL máis especializadas. Estas sentenzas SQL vanse a denominar sentenzas SQL precompiladas e van *recibir parámetros de entrada*.



Nesta imaxe mostrase a herdanza da interface `PreparedStatement`

O interface `PreparedStatement` herda do interface `Statement` e diferénciase dela de dous xeitos:

- As instancias de `PreparedStatement` conteñen sentenzas SQL que xa foron compiladas. Isto é o que fai a unha sentenza ser "*prepared*" (preparada).
- A sentenza SQL que contén un obxecto `PreparedStatement` pode conter un ou máis parámetros de entrada.

Un parámetro de entrada é aquel que o seu valor non se especifica cando a sentenza é creada, no seu lugar a sentenza vai ter un signo de interrogación (?) por cada parámetro de entrada.

Antes de executarse a sentenza, débese especificar un valor para cada un dos parámetros, a través dos métodos `setXXX` apropiados. Estes métodos `setXXX` engádeos a interface `PreparedStatement`.

Debido a que as sentenzas dos obxectos `PreparedStatement` están precompiladas, a súa execución será máis rápida que a dos obxectos `Statement`. Polo tanto, unha sentenza SQL que vai ser executada varias veces, adóitase creala como un obxecto `PreparedStatement` para gañar en eficiencia. Tamén se utilizará este tipo de sentenzas para pasarlle parámetros de entrada ás sentenzas SQL.

Ao herdar da interface `Statement`, a interface `PreparedStatement` herda todas as súas funcionalidades. Ademais, engade unha serie de métodos que permiten asignar un valor a cada un dos parámetros de entrada.

Creando obxectos `PreparedStatement`

Para crear un obxecto `PreparedStatement`, débese lanzar o método `prepareStatement()` da interface `Connection`, sobre o obxecto que representa a conexión establecida coa base de datos.

```

Connection conexion=DriverManager.getConnection(url,"Usuario","Clave");
PreparedStatement sentencia=conexion.prepareStatement("sentSQLconvalores?");
  
```

No seguinte exemplo, créase un obxecto `PreparedStatement` que representa unha sentenza SQL, con dous parámetros de entrada. O obxecto `sentenza` conterá a sentenza SQL indicada, que xa foi compilada para a súa execución e enviada ao SXBD.


```
Connection conexion=DriverManager.getConnection(url,"Usuario1","AAAAA");
PreparedStatement sentenza=conexion.prepareStatement
("UPDATE Taboa1 SET campo1=? WHERE campo2 =?");
```

Suministrar valores para os parámetros dun PreparedStatement

Antes de poder executar un obxecto PreparedStatement, débese asignar un valor para cada un dos seus parámetros. Isto realízase mediante a chamada ao método setXXX, onde XXX é o tipo de datos apropiado para o parámetro.

Si o valor que queremos substituír por unha marca de interrogación é un int de Java, podemos chamar ao método setInt. Si o valor que queremos substituír é un String de Java, podemos chamar ao método setString. Si é de tipo long, o método a utilizar será setLong, etc.

A sintaxe do método setXXX é:

```
setXXX(int posición, valor)
```

O primeiro argumento é a posición ordinal do parámetro ao que se lle vai a asignar o valor, e o segundo argumento é o valor a asignar.

Por exemplo, o código seguinte crea un obxecto PreparedStatement e asígnalle ao primeiro dos seus parámetros un valor de tipo String e ao segundo un valor de tipo int:

```
PreparedStatement sentenza=conexion.prepareStatement
("UPDATE Taboa1 SET campo1=? WHERE campo2 =?");
sentenza.setString(1,"Pepe");
sentenza.setInt(2,23);
```

O conector enviará á base de datos, a cadea “Pepe” como un JDBC VARCHAR (que é o mapeo estándar para un String Java) e o valor 23 como un JDBC INTEGER.

O programador debe asegurarse de que, o tipo de datos Java de cada parámetro sexa convertido a un tipo de datos JDBC compatible e esperado pola base de datos. Consideremos o caso no que a base de datos espera un tipo de datos SMALLINT, si se usa o método setByte, o conector enviará un JDBC TINYINT á base de datos, isto probablemente funcionará porque moitas bases de datos converten un tipo relacionado noutro, e de forma xeral un tipo TINYINT pode ser convertido nun SMALLINT, pero para facer aplicación máis portable e que se poda traballar con varios SXBD diferentes, é preferible que se usen tipos Java que se correspondan co tipo exacto JDBC esperado pola base de datos. Si o tipo esperado é SMALLINT, mellor usar setShort no lugar de setByte.

En xeral, hai un método setXXX para cada tipo Java. Algúns móstranse a continuación:

- void **setBoolean**(int posición, boolean x) throws SQLException

Asigna o valor x de tipo boolean ao parámetro. O conector convérteo a un tipo SQL BIT ou BOOLEAN cando se envía á base de datos.

- void **setByte** (int posición, byte x) throws SQLException

Asigna o valor x de tipo byte boolean ao parámetro. O conector convérteo a un tipo SQL TINYINT cando se envía á base de datos.

- void **setShort** (int posición, short x) throws SQLException

Asigna o valor x de tipo short ao parámetro. O conector convérteo a un tipo SQL SMALLINT cando se envía á base de datos.

- void **setInt** (int posición, int x) throws SQLException

Asigna o valor x de tipo int ao parámetro. O conector convérteo a un tipo SQL INTEGER cando se envía á base de datos.

- void **setLong** (int posición, long x) throws SQLException

Asigna o valor x de tipo long ao parámetro. O conector convérteo a un tipo SQL BIGINT cando se envía á base de datos.

- void **setFloat** (int posición, float x) throws SQLException

Asigna o valor *x* de tipo *float* ao parámetro *float*. O conector convérteo a un tipo SQL *REAL* cando se envía á base de datos.

- void **setDouble** (int posición, double *x*) throws *SQLException*
 - Asigna o valor *x* de tipo *double* ao parámetro. O conector convérteo a un tipo SQL *DOUBLE* cando se envía á base de datos.
- void **setBigDecimal** (int posición, *BigDecimal* *x*) throws *SQLException*
 - Asigna o valor *x* de tipo *java.math.BigDecimal* ao parámetro. O conector convérteo a un tipo SQL *NUMERIC* cando se envía á base de datos.
- void **setString** (int posición, *String* *x*) throws *SQLException*
 - Asigna o valor *x* de tipo *String* ao parámetro. O conector convérteo a un tipo SQL *VARCHAR* ou *LONGVARCHAR* (dependendo do tamaño do argumento segundo os límites dos valores *VARCHAR* do conector) cando se envía á base de datos.
- void **setBytes** (int posición, byte [] *x*) throws *SQLException*
 - Asigna o array de bytes ao parámetro. O conector convérteo a un tipo SQL *VARBINARY* ou *LONGVARCHAR* (dependendo do tamaño do argumento segundo os límites dos valores *VARCHAR* do conector) cando se envía á base de datos.
- void **setDate** (int posición, *Date* *x*) throws *SQLException*
 - Asigna o valor *x* de tipo *java.sql.Date* ao parámetro usando a zona horaria por defecto da máquina onde se executa a aplicación. O conector convérteo a un tipo SQL *DATE* cando se envía á base de datos.
- void **setTime** (int posición, *Time* *x*) throws *SQLException*
 - Asigna o valor *x* de tipo *java.sql.Time* ao parámetro. O conector convérteo a un tipo SQL *TIME* cando se envía á base de datos.
- void **setTimestamp** (int posición, *Timestamp* *x*) throws *SQLException*
 - Asigna o valor *x* de tipo *java.sql.Timestamp* ao parámetro. O conector convérteo a un tipo SQL *TIMESTAMP* cando se envía á base de datos.

Envío de parámetros moi grandes.

Os métodos *setBytes* e *setString* son capaces de enviar cantidades moi grandes de datos. Ás veces os programadores prefiren pasar os grandes bloques de datos, en pequenos anacos. Isto pode realizarse, asignado ao parámetro un fluxo de datos de entrada Java. Cando se executa a sentenza, o conector JDBC realizará repetidas chamadas a este fluxo de entrada, lendo os seus contidos e transmitíndoos á base de datos.

JDBC ten os seguintes métodos para asignar fluxos de entrada aos parámetros. Algunhas bases de datos necesitan coñecer o tamaño total dos datos a transferir antes de envialos, entón, débese especificar a lonxitude do fluxo de datos.

- void **setAsciiStream** (int posición, *InputStream* *x*, int lonxitude) throws *SQLException*
void **setAsciiStream** (int posición, *InputStream* *x*, long lonxitude) throws *SQLException*
void **setAsciiStream** (int posición, *InputStream* *x*) throws *SQLException*
Úsase para fluxos de entrada que conteñen caracteres ASCII. O conector JDBC fará calquera conversión necesaria de ASCII ao formato de caracteres de base de datos. O conector convérteo a un tipo SQL *LONGVARCHAR*.
- void **setBinaryStream**(int posición, *InputStream* *x*, int lonxitude) throws *SQLException*
void **setBinaryStream**(int posición, *InputStream* *x*, long lonxitude) throws *SQLException*
void **setBinaryStream** (int posición, *InputStream* *x*, int lonxitude) throws *SQLException*
Úsase para fluxos de entrada que conteñen bytes. O conector convérteo a un tipo SQL *LONGVARCHAR*.

- void **setCharacterStream**(int posición, Reader x, int lonxitude) throws SQLException
- void **setCharacterStream**(int posición, Reader x, long lonxitude) throws SQLException
- void **setCharacterStream**(int posición, Reader x) throws SQLException

Úsase para fluxos de entrada de caracteres que conteñen caracteres UNICODE. O conector convérteo a un tipo SQL LONGVARCHAR.

Estes métodos poden lanzar a excepción `SQLFeatureNotSupportedException`, si o controlador JDBC non é compatible co método.

O seguinte exemplo mostra como actualizar un campo cunha foto contida nun ficheiro. Cando a sentenza se executa, o fluxo de entrada `in` será chamado repetidamente ata completar os datos.

```
java.io.File ficheiro = new java.io.File("datos.png");
int lonxitude = file.length();
java.io.InputStream in = new java.io.FileInputStream (ficheiro);
java.sql.PreparedStatement sentenza = con.prepareStatement(
    "UPDATE Taboa SET campo5 =? WHERE clave=4");
sentenza.setBinaryStream (1, in, lonxitude);
sentenza.executeUpdate ();
```

Tamén podemos asignar aos parámetros valores moi grandes utilizando os métodos `SetXXX` que dan soporte aos tipos de datos SQL3. Estes métodos poden lanzar a excepción `SQLFeatureNotSupportedException`, si o controlador JDBC non soporta os tipos de datos SQL3.

Algúns destes métodos son:

- void **setBlob** (int posición, Blob x) throws SQLException

Asigna o valor `x` de tipo `java.sql.Blob` ao parámetro. O conector convérteo a un tipo SQL BLOB cando se envía á base de datos.

- void **setBlob** (int posición, InputStream x, long lonxitude) throws SQLException
- void **setBlob** (int posición, InputStream x) throws SQLException

Asigna o valor `x`, o cal é un fluxo de bytes de entrada, ao parámetro. O conector convérteo a un tipo SQL BLOB cando se envía á base de datos. Este método difire do `setBinaryStream`, xa que neste caso, o conector ten que facer traballo extra para determinar si os datos dos parámetros deben ser enviados como un LONGVARBINARY ou un BLOB.

- void **setClob** (int lonxitude, Clob x) throws SQLException
 - Asigna o valor `x` de tipo `java.sql.Clob` ao parámetro. O conector convérteo a un tipo SQL CLOB cando se envía á base de datos.
- void **setClob** (int posición, Reader x, long lonxitude) throws SQLException
- void **setClob** (int posición, Reader x) throws SQLException

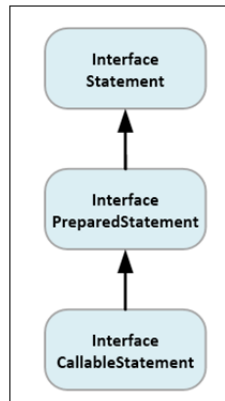
Asigna o valor `x`, o cal é un fluxo de caracteres de entrada, ao parámetro. O conector convérteo a un tipo SQL CLOB cando se envía á base de datos. Este método difire do `setCharacterStream`, xa que neste caso, o conector ten que facer traballo extra para determinar si os datos dos parámetros deben ser enviados como un LONGVARBINARY ou un CLOB.

5.3 Executar procedementos almacenados e funcións: Interface CallableStatement

Outro tipo de sentenzas que podemos utilizar en JDBC son as sentenzas `CallableStatement`. Esta interface ofrece a posibilidade de manexar parámetros de saída e de realizar chamadas a procedementos almacenados e funcións da base de datos, dunha forma estándar para todos os SXBD.

Un procedemento almacenado atópase dentro dunha base de datos; a chamada a un procedemento realízase mediante un obxecto `CallableStatement`. Antes de crear un obxecto `CallableStatement`, o programador deberá saber si o SXBD soporta procedementos almacenados.

A interface `CallableStatement` herda os métodos da interface `Statement`, que se encarga de sentenzas SQL xerais, e tamén os métodos da interface `PreparedStatement`, que se encarga de manexar parámetros de entrada.



Nesta imaxe mostrase a heranza da interface `CallableStatement`

Crear obxectos `CallableStatement`

A sintaxe para realizar a chamada a un procedemento almacenado é a seguinte:

- Si ten parámetros de entrada:

```
{call nome_do_procedemento[(?, ?, ...)]}
```

- Si devolve un parámetro de resultado (sería unha chamada a unha función):

```
{?=call nome_do_procedemento [(?.?.?.)]}
```

- A sintaxe dunha chamada a un procedemento sen ningún tipo de parámetros, sería:

```
{call nome_do_procedemento}
```

Os obxectos `CallableStatement` créanse co método `prepareCall` da interface `Connection`.

O seguinte exemplo crea unha instancia de `CallableStatement` que contén unha chamada ao procedemento almacenado `procl`, con dous argumentos e non devolve resultados.

```
CallableStatement sentenza = conexion.prepareCall( "{call procl(?, ?)}" );
```

Utilizar os parámetros de entrada, saída e de entrada/saída

Unha sentenza `CallableStatement` pode ter parámetros de entrada, de saída e de entrada/saída.

- Para pasarlle parámetros de entrada a un obxecto `CallableStatement`, utilízanse os métodos `setXXX` que herdaba do interface `PreparedStatement`.
- Si o procedemento almacenado devolve parámetros de saída, o tipo JDBC de cada parámetro de saída *debe ser rexistrado* antes de executar o obxecto `CallableStatement` correspondente.

Para rexistrar os tipos JDBC dos parámetros de saída, débese lanzar o método `CallableStatement.registerOutParameter`.

```
void registerOutParameter(int posición, int tipoSQL)
```

- posición: o primeiro parámetro é 1, o segundo é 2, e así sucesivamente.
- tipoSQL: é o tipo definido por `java.sql.Types`.

- Logo de rexistrar os parámetros, pódense *recuperar os valores* destes parámetros chamando ao método `getXXX` axeitado. Estes métodos son herdados da interface `Statement`, vistos anteriormente.

Executar o obxecto `CallableStatement`

`CallableStatement` soporta calquera chamada dos métodos de execución de `Statement` (`executeUpdate()`, `executeQuery()` ou `execute()`). O método máis flexible é `execute()`, xa que non necesita saber de antemán, si o procedemento almacenado retorna un conxunto de resultados ou non.

- **boolean `execute()` throws `SQLException`**

Executa o procedemento invocado no obxecto `CallableStatement` que pode ser calquera clase de sentenza SQL.

Devolve un booleano para indicar a forma do resultado. `True` si é un obxecto `ResultSet`, ou `False` si devolveu un contador de filas afectadas nunha sentenza de actualización.

Despois podemos invocar aos métodos `getResultSet` ou `getUpdateCount` para mostrar os resultados.

- **`ResultSet executeQuery()` throws `SQLException`**

Executa o procedemento invocado no obxecto `CallableStatement` cando se executa unha sentenza `SELECT` dentro do procedemento.

Devolve un `ResultSet`.

- **int `executeUpdate()` throws `SQLException`**

Executa o procedemento invocado no obxecto `CallableStatement` cando se executa unha sentenza `INSERT`, `DELETE`, `UPDATE` ou unha sentenza que non devolve ningún resultado, como unha sentenza DDL (`CREATE`).

Devolve un enteiro: o número de columnas afectadas na sentenza de actualización ou 0 si non devolve nada.

O seguinte exemplo mostra a chamada e execución dun procedemento almacenado, con nome `Procedemento1`, que ten dous parámetros, o primeiro de tipo enteiro de entrada e o segundo de tipo decimal de saída.

```
Connection conexion=DriverManager.getConnection(url);
CallableStatement sentenza=conexion.prepareCall("
    {call Procedemento1(?,?)}");
senteza.setInt(1,7); //parámetro de entrada, dámoslle o valor 7
senteza.registerOutParameter(2,java.sql.Types.DECIMAL);
//o parámetro dous é de saída (OUT)
senteza.execute(); //chamar ao procedemento
System.out.println("resultado"+ sentenza.getBigDecimal(2));
```

Xestión do `ResultSet`

En JDBC 1.0 os `ResultSet` só se poden percorrer de arriba abaixo e ademais, son de só lectura. JDBC 2.0 permite que os `ResultSet` se poidan percorrer en ambos sentidos, é dicir, permite situarse nun rexistro concreto e introduce posibilidades de actualización (concorrenia).

Os obxectos `ResultSet` poden ter distintas funcionalidades e características. Estas características son tipo, concorrencia e permanencia (holdability).

Tipos de `ResultSet`

Un `ResultSet` pode ser de tres tipos que se configuran coas seguintes constantes:

- **static int `TYPE_FORWARD_ONLY`**
 - O obxecto cursor só se pode mover cara a adiante. Opción por defecto.
- **static int `TYPE_SCROLL_INSENSITIVE`**

O cursor pódese mover en calquera dirección, pero é insensible aos cambios efectuados pola mesma transacción, ou por outras transaccións finalizadas con éxito (COMMIT).

▪ static int TYPE_SCROLL_SENSITIVE

O cursor pódese mover en calquera dirección e é sensible aos cambios nos datos do ResultSet.

Podemos comprobar, a través do método `DatabaseMetaData.supportsResultSetType` (TipoResultSet), o tipo de ResultSet que soporta o conector JDBC. No seguinte exemplo, compróbase si o conector JDBC de MySQL soporta cada tipo de ResultSet.

```
Connection conexion = DriverManager.getConnection(
    "jdbc:mysql://localhost:8888/Empresa","user1", "abc123.");
DatabaseMetaData md = conexion.getMetaData(); // obtén os metadatos
// Verifica os tipos de ResultSet
System.out.println("Soporta cursor TYPE_FORWARD_ONLY: "
    + md.supportsResultSetType(ResultSet.TYPE_FORWARD_ONLY));
System.out.println("Soporta cursor TYPE_SCROLL_INSENSITIVE: "
    + md.supportsResultSetType(ResultSet.TYPE_SCROLL_INSENSITIVE));
System.out.println("Soporta cursor TYPE_SCROLL_SENSITIVE: "
    + md.supportsResultSetType(ResultSet.TYPE_SCROLL_SENSITIVE));
```

Concurrencia

A concurrencia determina si o ResultSet pode actualizarse. Os valores de concurrencia dispoñibles son os seguintes:

▪ static int CONCUR_READ_ONLY

Indica que si se modifica unha columna dun obxecto ResultSet, non se verá reflectido na base de datos. É o valor por defecto.

▪ static int CONCUR_UPDATABLE

Indica que si se modifica unha columna dun obxecto ResultSet, verase reflectido na base de datos.

Nota: Non todos os conectores JDBC e SXBD soportan concurrencia. Para comprobalo pódese utilizar o método `supportsResultSetConcurrency()` da interface `DatabaseMetaData`. Este método retorna true se soporta o tipo de concurrencia especificado e false en caso contrario. Tamén o podemos verificar a través do método `DatabaseMetaData.supportsResultSetType` (TipoResultSet).

Exemplo:

```
Connection conexion = DriverManager.getConnection(
    "jdbc:mysql://localhost:8888/Empresa","user1", "abc123.");
DatabaseMetaData md = conexion.getMetaData(); // obtén os metadatos
// Verifica a concurrencia do ResultSet
System.out.println("Soporta o cursor CONCUR_READ_ONLY: "
    + md.supportsResultSetType(ResultSet.CONCUR_READ_ONLY));
System.out.println("Soporta o cursor CONCUR_UPDATABLE: "
    + md.supportsResultSetType(ResultSet.CONCUR_UPDATABLE));
```

Permanencia do cursor (holdability)

Permite controlar o comportamento dos obxectos ResultSet cando se chama ó método `Connection.commit()`. Este método pode pechar ou non os obxectos ResultSet que se crearon durante a transacción actual. A propiedade holdability do ResultSet dá control á aplicación sobre si os obxectos ResultSet se pechan ou non, cando é realizado un commit.

As seguintes constantes ResultSet poden subministrarse aos métodos `createStatement`, `prepareStatement` e `prepareCall`:

▪ **HOLD_CURSORS_OVER_COMMIT**

Os cursores `ResultSet` non se pechan, mantéñense abertos cando é chamado o método `commit`. Estes cursores son ideais cando a aplicación usa maiormente obxectos `ResultSet` de só lectura.

▪ **CLOSE_CURSORS_AT_COMMIT**

Os obxectos `ResultSet` (cursores) péchanse cando se chama ao método `commit`. Isto da mellor rendemento a algunhas aplicacións.

O valor por defecto da permanencia varía dependendo do SXBD. Non todas as bases de datos e conectadores JDBC son compatibles con estes `ResultSet`. Podemos comprobar a través do método `DatabaseMetaData.supportsResultSetType(resultSetType)` ou mediante o método `DatabaseMetaData.supportsResultSetHoldability(tipoPermanencia)`.

Por exemplo:

```
Connection conexion = DriverManager.getConnection(
    jdbc:mysql://localhost:8888/Empresa","user1", "abc123.");
DatabaseMetaData md = conexion.getMetaData(); // obtén os metadatos

// verifica a permanencia do ResultSet
// obtén a permanencia por defecto
System.out.println("permanencia por defecto: "+md.getResultSetHoldability());
System.out.println("¿soporta HOLD_CURSORS_OVER_COMMIT? " +
md.supportsResultSetHoldability(ResultSet.HOLD_CURSORS_OVER_COMMIT));
System.out.println("¿soporta CLOSE_CURSORS_AT_COMMIT? " +
md.supportsResultSetHoldability(ResultSet.CLOSE_CURSORS_AT_COMMIT));
```

Establecer as características do ResultSet

Para obter un `ResultSet` dalgún destes tipos, especificase o seu tipo na chamada dos métodos `createStatement`, `prepareStatement` ou `prepareCall` do obxecto `Connection`, tal como se indica a continuación:

```
createStatement(int tipoResultSet, int tipoConcorrenca);
prepareStatement(String sql, int tipoResultSet, int tipoConcorrenca);
prepareCall(String sql, int tipoResultSet, int tipoConcorrenca);
```

Parámetros:

O tipo de `ResultSet` pode ser:

`ResultSet.TYPE_FORWARD_ONLY`, `ResultSet.TYPE_SCROLL_INSENSITIVE` ou `ResultSet.TYPE_SCROLL_SENSITIVE`.

O tipo de concorrencia pode ser:

`ResultSet.CONCUR_READ_ONLY` ou `ResultSet.CONCUR_UPDATABLE`.

Neste caso, non se especificou a permanencia (`holdability`) do `ResultSet`, polo tanto, será a que teña implementada por defecto o SXBD, pero si se quere expresar explicitamente, pódese indicar cun parámetro na chamada dos métodos `createStatement`, `prepareStatement` ou `prepareCall` do obxecto `Connection`, como se indica a continuación:

```
createStatement(int tipoResultSet, int tipoConcorrenca, int tipoPermanencia);
prepareStatement(String sql, int tipoResultSet, int tipoConcorrenca,
    int tipoPermanencia);
prepareCall(String sql, int tipoResultSet, int tipoConcorrenca,
    int tipoPermanencia);
```

✓ Parámetros:

- tipo de permanencia pode ser: `HOLD_CURSORS_OVER_COMMIT` ou `CLOSE_CURSORS_AT_COMMIT`.

Métodos para moverse no ResultSet

Para acceder aos datos do conxunto de resultados devoltos por unha consulta, o obxecto `ResultSet` mantén un cursor, que apunta a unha fila do obxecto `ResultSet`, denominada fila actual. Cando se crea un obxecto `ResultSet`, o cursor queda colocado antes da primeira fila.

Os métodos dispoñibles para mover o cursor son:

- **boolean `absolute`(int fila)**
Move o cursor á fila especificada.
- **void `afterLast`()**
Move o cursor ao final, xusto antes da última fila. Se o obxecto `ResultSet` non contén filas, este método non ten efecto.
- **void `beforeFirst`()**
Move o cursor ao principio do obxecto `ResultSet`, antes da primeira fila. Se o obxecto `ResultSet` non contén filas, este método non ten efecto.
- **boolean `next`()**
Move o cursor á posición seguinte. Retorna `true` se o cursor queda apuntando a unha fila e `false` se o cursor apunta detrás da última fila.
- **boolean `previous`()**
Move o cursor á posición anterior. Retorna `true` se o cursor apunta a unha fila e `false` se o cursor apunta antes da primeira fila.
- **boolean `relative`(int filas)**
Move o cursor, desde a posición actual, tantas posicións como se indique no parámetro. Pode ser positivo (move o cursor cara adiante) ou negativo (move o cursor cara atrás). O intento de ir máis aló da primeira fila ou última no conxunto de resultados, sitúa o cursor antes da fila da primeira ou logo da última.
- **boolean `first`()**
Move o cursor á primeira fila. Retorna `true` si o cursor queda apuntando á primeira fila e `false` se o obxecto `ResultSet` non contén filas.
- **boolean `last`()**
Move o cursor á última fila. Retorna `true` si o cursor queda apuntando a última fila `false` se o obxecto `ResultSet` non contén filas.
- **int `getRow`()**
 - Devolve a posición onde está o cursor.

Ademais, existen os seguintes métodos, que devolven un valor `boolean` para preguntar pola posición do cursor:

- **boolean `isBeforeFirst`()**: devolve `true` si se está antes do primeiro rexistro.
- **boolean `isAfterLast`()**: devolve `true` si se está tralo último rexistro.
- **boolean `isFirst`()**: devolve `true` si se está no primeiro rexistro.
- **boolean `isLast`()**: devolve `true` si se está no último rexistro.

Hai que recordar, que a configuración por defecto dun `ResultSet` é `TYPE_FORWARD_ONLY`, o que significa que non se pode facer `scroll`; non se pode chamar a outros métodos que non sexan `next()`.

Métodos para traballar dinamicamente co ResultSet

- **void `updateXXX`(String columna, XXX valor)**
 - Modifica a columna co valor de tipo XXX especificado no parámetro valor.

- **void updateRow()**
 - Modifica a fila da base de datos cos contidos da fila actual do ResultSet.
- **void moveToInsertRow()**
 - Móvese a unha fila especial que se utiliza para inserir unha fila de datos nova na táboa. Si se move o cursor, a unha fila distinta antes de chamar a `insertRow()`, a base de datos non se actualizará.
- **void insertRow()**
 - Insire na base de datos o contido da fila actual do ResultSet.
- **void moveToCurrentRow()**
 - Move o cursor á posición que estaba antes de que o método `moveToInsertRow()` fose chamado.
- **void cancelRowUpdates()**
 - Cancela os cambios feitos na fila.
- **void deleteRow()**
 - Borra a fila actual do ResultSet e da base de datos.

Exemplo:

```
Statement stmt = conexion.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                                           ResultSet.CONCUR_UPDATABLE);
...
ResultSet rst = stmt.executeQuery("select * from empregados");
...
// Actualiza unha fila
rst.absolute(2); // move o cursor á fila 2
rst.updateString("NOME", "Luis"); // cambio nome do empregado da fila 2
rst.updateRow(); // actualiza a base de datos
rst.moveToInsertRow(); // move o cursor á fila de inserción
// Crea unha fila nova
rst.updateInt("CODIGO", 9); // damos valores ás columnas
rst.updateString("NOME", "Pepe");
rst.updateString("APELLIDO1", "García");
rst.updateString("APELLIDO2", "Campos");
rst.insertRow(); // inserimos a fila na base de datos
rst.moveToCurrentRow();
```

6. Acceso aos Metadatos

Ás veces é necesario obter información sobre o proveedor, conxunto de filas, táboa, columnas ou outra información de bases de datos. Os datos sobre da estrutura de base de datos denomínanse metadatos.

O acceso aos metadatos evita que o programador teña que coñecer a fondo a organización da base de datos. Esta información non é de interese para o usuario pero si que é necesaria para o programador para manexar os datos.

Existen dous interfaces en java.sql para os metadatos:

- **ResultSetMetaData**. Proporciona información do tipo e propiedades de cada columna dun obxecto ResultSet.
- **DatabaseMetaData**. Proporciona información dunha base de datos ou un SXBD particular.

6.1 Interface DatabaseMetaData

Esta interface ofrece información sobre a base de datos coa que se estableceu unha conexión.

Para obter esta información, esta interface aporta un gran número de métodos diferentes. Moitos destes métodos devolven obxectos `ResultSet` contendo a información correspondente, polo tanto, deberemos usar os métodos `getXXX` para recuperar a información.

Si un conector non soporta un método do interface `DatabaseMetaData` lanzarase unha excepción `SQLException`, e no caso de que o método devolva un obxecto `ResultSet`, obterase un `ResultSet` baleiro ou lanzarase unha excepción `SQLException`.

Para obter un obxecto `DatabaseMetaData` sobre o que lanzar os métodos que nos darán a información sobre o SXBD, débese lanzar o método `getMetaData()` sobre o obxecto `Connection` que se corresponda coa conexión á base de datos da que queremos obter a información, como se pode observar no código seguinte:

```
Connection conexion=DriverManager.getConnection(url);
DatabaseMetaData datosdb=conexion.getMetaData();
```

Todos os métodos da interface `DatabaseMetaData` lanzan excepcións `SQLException`. Implementa máis de 150 métodos diferentes que nos ofrecen unha información detallada sobre a base de datos.



Na páxina: <http://docs.oracle.com/javase/7/docs/api/java/sql/DatabaseMetaData.html>. pódese obter máis información dos métodos da interface `DatabaseMetaData`.

Algúns métodos son os seguintes:

Métodos que mostran información sobre o SXBD, usuario actual conectado, conector, URL e base de datos

- String ***getDatabaseProductName()*** throws `SQLException`
 - Recupera o nome do SXBD.
- String ***getDatabaseProductVersion()*** throws `SQLException`
 - Recupera o número de versión do SXBD.
- int ***getDatabaseMajorVersion()***
 - Recupera o número de versión principal do SXBD.
- int ***getDatabaseMinorVersion()***
 - Recupera o número de versión secundario do SXBD.
- String ***getDriverName()*** throws `SQLException`
 - Recupera o nome do controlador JDBC utilizado.
- int ***getDriverMajorVersion()***
 - Recupera o número da versión principal do conector JDBC.
- int ***getDriverMinorVersion()***
 - Recupera o número da versión secundaria do conector JDBC.
- String ***getDriverVersion()*** throws `SQLException`
 - Recupera o número de versión do conector JDBC utilizado.
- String ***getURL()*** throws `SQLException`
 - Devolve o URL da base de datos ou NULL si non o pode atopar.
- String ***getUserName()*** throws `SQLException`
 - Devolve o nome do usuario actual conectado á base de datos.
- booleano ***IsReadOnly()*** throws `SQLException`
 - Devolve true si a base de datos é de só lectura.

Patróns e caracteres comodíns

Nalgúns métodos que obteñen información das táboas, columnas, claves primarias, claves foráneas, etc, nos seus parámetros pódese utilizar un *patrón*, na cadea onde se especifica o *esquema e as táboas*.

Nos patróns utilízanse os caracteres comodíns: "_" ou "%" para delimitar as procuras.

- O carácter '_' representa calquera carácter individual.
- O carácter '%' representa calquera secuencia de cero ou máis caracteres.

Por exemplo: si na cadea que especifica unha táboa, poñemos "person%", obteríamos todas as táboas que o nome empeza por "person".

Métodos para obter información sobre as táboas, columnas, claves primarias, foráneas, ...

- ResultSet **getTables** (String catálogo, String esquemaPatron, String táboaPatron, String [] tipoTáboas) throws SQLException.

Recupera unha descrición das táboas dispoñibles no catálogo dado.

Os tipos de táboas son: "TABLE", "VIEW", "SYSTEM TABLE", "GLOBAL TEMPORARY", "LOCAL TEMPORARY", "ALIAS", "SYNONYM". Ao poñer null, devolveranos todos os tipos de táboas.

No argumento catálogo ou esquema pódese poñer null. Neste caso referímonos ao actual.

As columnas do ResultSet devoltas de uso máis común son:

- 1. TABLE_CAT String => Catálogo da táboa (pode ser nulo).
- 2. TABLE_SCHEM String => Esquema da táboa (pode ser nulo).
- 3. TABLE_NAME String => Nome da táboa.
- 4. TABLE_TYPE String => Tipo de táboa
- 5. REMARKS String => Comentario sobre a táboa.

Exemplo:

```
System.out.println(">>>Táboas existentes:");
String patron = "%"; //listamos todas as táboas que comecen por C
String tipos[] = new String[2];
tipos[0] = "TABLE"; //táboas de usuario
tipos[1] = "SYSTEM TABLE"; //táboas do sistema
ResultSet taboas = metadatos.getTables(null, null, patron, tipos);
while (taboas.next()) {
    //Por cada táboa obtemos o seu nome e tipo
    System.out.println(" Nome:" + taboas.getString("TABLE_NAME")+
        " Tipo:" + taboas.getString("TABLE_TYPE")+ " Esquema:" +
        taboas.getString("TABLE_SCHEM")+
        " Catalogo: " + taboas.getString("TABLE_CAT"));
}
```

- ResultSet **getPrimaryKeys** (String catálogo, String esquema, String táboa) throws SQLException

Recupera información das columnas de clave principal da táboa dada.

Os parámetros catálogo ou esquema poden ter o valor NULL, entón o nome do catálogo ou do esquema non se utilizan para delimitar a procura.

As columnas do ResultSet devoltas de uso máis común son:

- 1. TABLE_CAT String => Catálogo da táboa (pode ser nulo).
- 2. TABLE_SCHEM String = Esquema da táboa (pode ser nulo).
- 3. TABLE_NAME String => Nome da táboa.
- 4. COLUMN_NAME String => Nome da columna.
- 5. KEY_SEQ Short => Número de secuencia dentro da clave principal (un valor de 1 representa a primeira columna da clave primaria, un valor de 2 representaría a segunda columna na clave principal e así sucesivamente).
- 6. PK_NAME String => Nome da clave principal (pode ser nulo).

- `ResultSet` ***getColumns*** (String catálogo, String esquemaPatron, String taboaPatron, String columnaPatron) throws `SQLException`

Recupera unha descrición das columnas da táboa dispoñibles no catálogo.

Os parámetros catálogo ou esquema poden ter o valor `NULL`, entón o nome do catálogo ou do esquema non se utilizan para delimitar a procura.

Algunhas columnas do `ResultSet` son:

- 1. `TABLE_CAT` String => Catálogo da táboa (pode ser nulo).
- 2. `TABLE_SCHEM` String => Esquema da táboa (pode ser nulo).
- 3. `TABLE_NAME` String => Nome da táboa.
- 4. `COLUMN_NAME` String => Nome da columna.
- 5. `DATA_TYPE` int => Tipo de datos SQL (`java.sql.Types`). Devolve un enteiro.
- 6. `TYPE_NAME` String => Nome do tipo de Datos.
- 7. `COLUMN_SIZE` int => Tamaño da columna.
- 9. `DECIMAL_DIGITS` int => O número de díxitos decimais. Devólvese `NULL` para tipos de datos onde `DECIMAL_DIGITS` non é aplicable.
- 11. `NULLABLE` int => Acepta valores `NULL`.
 - `columnNoNulls` - non pode permitir valores `NULL`.
 - `columnNullable` - permite valores `NULL`.
 - `columnNullableUnknown` - descoñecida a nulabilidade.
- 12. `REMARKS` String => Comentario que describe a columna (pode ser nulo).
- 13. `COLUMN_DEF` String => Valor predeterminado da columna (pode ser nulo).
- 18. `IS_NULLABLE` String => Determinar si unha columna acepta valores `NULL` utilizando as Normas ISO.
 - YES --- Si a columna pode incluír valores `NULL`.
 - NON --- Si a columna non pode conter valores `NULL`.
 - Cadea baleira --- Si a aceptación de valores `NULL` para a columna se descoñece.
- 23. `IS_AUTOINCREMENT` String => Indica si esta columna se incrementa de modo automático.
 - YES --- Si a columna se incrementa de forma automática.
 - NO --- Si a columna non é autoincrementada.
 - Cadea baleira --- si non se pode determinar si a columna é autoincrementada.
- 24. `IS_GENERATEDCOLUMN` String => Indica si se trata dunha columna xerada.
 - SI --- Si esta columna é xerada.
 - NON --- Si non é unha columna xerada.
 - Cadea baleira si non se pode determinar si se trata dunha columna xerada.

- `ResultSet` ***getExportedKeys*** (String catálogo, String esquema, String táboa) throws `SQLException`.

Devolve unha lista de todas as claves foráneas que fan referencia á clave principal da táboa dada.

Os parámetros catálogo ou esquema poden ter o valor `NULL`, entón o nome do catálogo ou do esquema non se utilizan para delimitar a procura.

A descrición dalgunhas columnas do `ResultSet` son:

- 1. `PKTABLE_CAT` String => Catálogo da táboa da clave principal que se referencia (pode ser nulo).
- 2. `PKTABLE_SCHEM` String => Esquema da táboa da clave principal que se referencia (pode ser nulo).
- 3. `PKTABLE_NAME` String => Nome da táboa.
- 4. `PKCOLUMN_NAME` String => Nome da columna da clave principal que se referencia.
- 5. `FKTABLE_CAT` String => Catálogo da táboa da clave externa (pode ser nulo).
- 6. `FKTABLE_SCHEM` String => Esquema da táboa da clave externa (pode ser nulo).

- 7. FKTABLE_NAME String => Nome da táboa da clave externa.
 - 8. FKCOLUMN_NAME String => Nome da columna da clave externa.
 - 9. KEY_SEQ short => Número de secuencia dentro dunha clave externa (un valor de 1 representa a primeira columna da clave externa, un valor de 2 representaría a segunda columna na clave externa).
 - 10. UPDATE_RULE short => Acción que se leva a cabo na clave externa cando se actualiza a clave primaria á que referencia.
 - importedNoAction: non se permite a actualización da clave primaria.
 - importedKeyCascade: cambio en cascada da clave externa.
 - importedKeySetNull: cambia a clave externa a NULL.
 - importedKeySetDefault: cambia a clave externa ao valor predeterminado.
 - importedKeyRestrict: igual que importedKeyNoAction (por compatibilidade ODBC 2.x).
 - 11. DELETE_RULE short => A acción que se leva a cabo na clave externa cando se elimina a clave primaria á que referencia.
 - importedKeyNoAction: non se permite borrar a clave primaria.
 - importedKeyCascade: borrado en cascada das filas da clave externa que referencia á clave primaria eliminada.
 - importedKeySetNull: clave externa a NULL.
 - importedKeyRestrict : igual que importedKeyNoAction (por compatibilidade ODBC 2.x).
 - importedKeySetDefault: Clave externa co valor por defecto.
 - 12. FK_NAME String => Nome da clave externa (pode ser nulo).
 - 13. PK_NAME String => Nome da clave principal (pode ser nulo).
- ResultSet **getImportedKeys** (String catálogo, String esquema, String táboa) throws SQLException.
- Devolve as claves externas na táboa dada.
- Os parámetros catálogo ou esquema poden ter o valor NULL, entón o nome do catálogo ou do esquema non se utilizan para delimitar a procura.
- A descrición dalgunhas columnas do ResultSet son:
- 1. PKTABLE_CAT String => Catálogo da táboa que contén a clave principal á que se fai referencia (pode ser nulo).
 - 2. PKTABLE_SCHEM String => Esquema da táboa que contén a clave principal á que se fai referencia (pode ser nulo).
 - 3. PKTABLE_NAME String => Nome da táboa que contén a clave principal á que se fai referencia.
 - 4. PKCOLUMN_NAME String => Nome da columna de clave principal que se referencia.
 - 5. FKTABLE_CAT String => Catálogo da táboa da clave externa (pode ser nulo).
 - 6. FKTABLE_SCHEM String => Esquema da táboa da clave externa (pode ser nulo).
 - 7. FKTABLE_NAME String => Nome da táboa da clave externa.
 - 8. FKCOLUMN_NAME String => Nome de columna de clave externa.
 - 9. KEY_SEQ short => Número de secuencia dentro dunha clave externa (un valor de 1 representa a primeira columna da clave externa, un valor de 2 representa a segunda columna na clave externa).
 - 10. UPDATE_RULE short => A acción que se leva a cabo na clave externa cando se actualiza a clave primaria á que referencia.
 - importedNoAction: non se permite a actualización da clave primaria.
 - importedKeyCascade: cambio en cascada da clave externa.
 - importedKeySetNull: cambia a clave externa a NULL.
 - importedKeySetDefault: cambia a clave externa ao valor predeterminado.

- importedKeyRestrict: igual que importedKeyNoAction (por compatibilidade ODBC 2.x).
 - 11. DELETE_RULE short => A acción que se leva a cabo na clave externa cando se elimina a clave primaria á que referencia.
 - importedKeyNoAction: non se permite borrar a clave primaria.
 - importedKeyCascade: borrado en cascada das filas da clave externa que referencia á clave primaria eliminada.
 - importedKeySetNull: clave externa a NULL.
 - importedKeyRestrict : igual que importedKeyNoAction (por compatibilidade ODBC 2.x).
 - importedKeySetDefault: clave externa co valor por defecto.
 - 12. FK_NAME String => Nome da clave externa (pode ser nulo).
 - 13. PK_NAME String => Nome da clave principal (pode ser nulo).
- **ResultSet *getProcedures*** (String catalogo, esquemaPatron, String procedimientoPatron) throws SQLException.
- Recupera unha descrición dos procedementos almacenados dispoñibles no catálogo dado.
 - Os parámetros catálogo ou esquema poden ter o valor NULL, entón o nome do catálogo ou do esquema non se utilizan para delimitar a procura.
 - As columnas do `ResultSet` son:
 - 1. PROCEDURE_CAT String => Catálogo do procedemento (pode ser null).
 - 2. PROCEDURE_SCHEM String => Esquema do procedemento (pode ser null).
 - 3. PROCEDURE_NAME String => Nome do procedemento.
 - 4. Reservado para uso futuro.
 - 5. Reservado para uso futuro.
 - 6. Reservado para uso futuro.
 - 7. REMARKS String => Comentarios do procedemento.
 - 8. PROCEDURE_TYPE short => Clase de procedemento.
 - ProcedureResultUnknown: non determina si devolve un valor.
 - ProcedureNoResult: non devolve un valor de retorno.
 - ProcedureReturnsResult: devolve un valor de retorno.
- **ResultSet *getProcedureColumns***(String catálogo, String catalogo, esquemaPatron, String procedimientoPatron, String columnaPatron) throws SQLException.
- Devolve unha descrición dos parámetros de entrada e de saída dos procedementos dun catálogo.
- A descrición dalgunhas columnas do `ResultSet` é:
- 1. PROCEDURE_CAT String => Catálogo do procedemento (pode ser null).
 - 2. PROCEDURE_SCHEM String => Esquema do procedemento (pode ser null).
 - 3. PROCEDURE_NAME String => Nome do procedemento.
 - 4. COLUMN_NAME String => Nome do parámetro.
 - 5. COLUMN_TYPE Short => Tipo do parámetro:
 - ProcedureColumnUnknown: non se pode determinar.
 - ProcedureColumnIn : parámetro de entrada.
 - ProcedureColumnInOut: parámetro de entrada/saída.
 - ProcedureColumnOut: parámetro de saída.
 - ProcedureColumnReturn: o procedemento devolve un valor (función).
 - ProcedureColumnResult: o procedemento devolve un `ResultSet`.
 - 6. DATA_TYPE int => tipo de datos SQL (java.sql.Types).
 - 8. PRECISION int => precisión.

- 9. LENGTH int => lonxitude en bytes dos datos.
- 10. SCALE short => escala. Devólvese NULL para tipos de datos onde a escala non é aplicable.
- 12. NULLABLE short => pode conter NULL.
 - ProcedureNoNulls - non permite valores NULL.
 - ProcedureNullable - permite valores NULL.
 - ProcedureNullableUnknown - nulabilidade descoñecida.
- 13. REMARKS String => Comentario do parámetro.
- 14. COLUMN_DEF String => Valor predeterminado do parámetro (pode ser nulo).
- 18. ORDINAL_POSITION int => A posición ordinal, a partir de 1, que ocupa a columna no conxunto do `ResultSet`. Si se trata dunha columna que almacena o valor devolto polo procedemento, o seu valor é 0.
- 19. IS_NULLABLE String => Determinar si o parámetro acepta valores NULL utilizando as normas ISO.
 - YES: si o parámetro pode incluír valores NULL.
 - NO: si o parámetro non pode conter valores NULL.
 - Cadea baleira: si a aceptación de valores NULL para a columna se descoñece.
- 20. SPECIFIC_NAME String => o nome que identifica de xeito único este procedemento dentro do seu esquema.

■ **ResultSet *getCatalogs()* throws SQLException.**

Devolve unha lista de catálogos de información na base de datos.

O `ResultSet` só ten unha columna:

1. TABLE_CATALOG String => Nome do catálogo.

■ **ResultSet *getSchemas()* throws SQLException.**

`ResultSet getSchemas (String catalogo, String esquemaPatron)` throws SQLException

Recupera os nomes dos esquemas dispoñibles no catálogo.

As columnas do `ResultSet` son:

- 1. TABLE_SCHEM String => Nome do esquema.
- 2. TABLE_CATALOG String => Nome do catálogo (pode ser nulo).

■ **ResultSet *getTableTypes()* throws SQLException.**

Recupera os tipos de táboas dispoñibles na base de datos.

O `ResultSet` só ten unha columna:

- 1.- TABLE_TYPE String: Os tipos de táboas son: "TABLE", "VIEW", "SYSTEM TABLE", "GLOBAL TEMPORARY", "LOCAL TEMPORARY", "ALIAS", "SYNONYM".

■ **ResultSet *GetTypeInfo()*.**

Recupera unha descrición de todos os tipos de datos soportados pola base de datos.

■ **ResultSet *getTablePrivileges*(String catálogo, String esquemaPatron, String tablaPatron) throws SQLException.**

Devolve unha descrición dos privilexios para acceder á táboa en cuestión. Isto non implica que sexan estes os privilexios de todas as columnas.

A descrición dalgunhas columnas do `ResultSet` é:

- 1. TABLE_CAT String => Catálogo da táboa (pode ser nulo).
- 2. TABLE_SCHEM String => Esquema da táboa (pode ser nulo).
- 3. TABLE_NAME String => Nome da táboa.
- 4. GRANTOR String => Otorgante do permiso (pode ser nulo).
- 5. GRANTEE String => Usuario que recibe o permiso.
- PRIVILEGE String => Nome do permiso (SELECT, INSERT, UPDATE, ...).

- IS_GRANTABLE String => Si o valor é "YES" permítese ao usuario que recibe o permiso concedelo aos demais, si é "NON" non se permite e si é "NULL" non se coñece o privilexio.

Métodos para obter información sobre as funcións, procedementos e características dispoñibles

- String **getCatalogSeparator()** throws SQLException.
Recupera a cadea que se utiliza como separador entre o catálogo e o nome da táboa.
- String **getSQLKeywords()** throws SQLException
Recupera unha lista de todas as palabras reservadas de SQL.
- String **getIdentifierQuoteString()** throws SQLException
Recupera a cadea que se utiliza para delimitar os identificadores SQL.
- String **getSearchStringEscape()** throws SQLException
Recupera a cadea que se pode utilizar para escapar caracteres comodíns.
- String **getNumericFunctions()** throws SQLException
Recupera unha lista das funcións matemáticas dispoñibles.
- String **getStringFunctions()** throws SQLException
Recupera unha lista das funcións de cadeas dispoñibles.
- String **getTimeDateFunctions()** throws SQLException
Recupera unha lista das funcións de data e hora dispoñibles.
- String **getSystemFunctions()** throws SQLException
Recupera unha lista das funcións do sistema dispoñibles.
- boolean **allProceduresAreCallable()** throws SQLException
Recupera si o usuario actual pode chamar a todos os procedementos que devolven o método getProcedures().
- boolean **allTablesAreSelectable()** throws SQLException
Recupera si o usuario actual pode utilizar todas as táboas devoltas polo método getTables() nunha sentença SELECT.

Métodos para obter información dos límites do conector

Outro grupo de métodos proporciona os límites impostos por un determinado conector. Algúns dos métodos desta categoría son:

- int **getMaxColumnNameLength()** throws SQLException
Devolve o número máximo de caracteres que pode ter o nome dunha columna.
- int **getMaxColumnsInGroupBy()** throws SQLException
Devolve o máximo número de columnas que poden aparecer nunha cláusula GROUP BY.
- int **getMaxColumnsInIndex()** throws SQLException
Devolve o máximo número de columnas nun índice.
- int **getMaxColumnsInOrderBy()** throws SQLException
Devolve o máximo número de columnas que poden aparecer nunha cláusula ORDER BY.
- int **getMaxColumnsInSelect()** throws SQLException
Devolve o máximo número de columnas que poden aparecer nunha consulta SELECT.
- int **getMaxColumnsInTable()** throws SQLException
Devolve o máximo número de columnas que poden aparecer nunha táboa.

- `int getMaxConnections()` throws `SQLException`
Devolve o máximo número de conexións simultáneas á base de datos. Un resultado de cero significa que non hai límite ou que o límite non é coñecido.
- `int getMaxCursorNameLength()` throws `SQLException`
Devolve o número máximo de caracteres dun cursor.
- `int getMaxIndexLength()` throws `SQLException`
Devolve o máximo número de bytes dun índice.
- `int getMaxSchemaNameLength()` throws `SQLException`
Devolve a lonxitude máxima dun nome de esquema.
- `int getMaxProcedureNameLength()` throws `SQLException`
Devolve a lonxitude máxima dun nome de procedemento.
- `int getMaxCatalogNameLength()` throws `SQLException`
Devolve a lonxitude máxima dun nome de catálogo.
- `int getMaxRowSize()` throws `SQLException`
 - Devolve a lonxitude máxima dunha fila.
- `int getMaxStatementLength()` throws `SQLException`
Devolve a lonxitude máxima dunha sentenza SQL.
- `int getMaxStatements()` throws `SQLException`
Devolve o máximo número de sentenzas simultáneas que soporta a base de datos. Un resultado de cero significa que non hai límite ou que o límite non se sabe.
- `int getMaxTableNameLength()` throws `SQLException`
Devolve a lonxitude máxima do nome dunha táboa.
- `int getMaxTablesInSelect()` throws `SQLException`
Devolve o número máximo de táboas nunha consulta SELECT.
- `int getMaxUserNameLength()` throws `SQLException`
Devolve a lonxitude máxima do nome de usuario.

Métodos para obter información do soporte de transaccións

Hai un pequeno grupo de métodos que proporcionan información sobre as características das transaccións soportadas polo conector. Son exemplos desta categoría:

- `boolean supportsTransactions()` throws `SQLException`
Devolve true si o sistema de bases de datos soporta as transaccións.
- `int getDefaultTransactionIsolation()` throws `SQLException`
Devolve o nivel de illamento de transaccións predeterminado.
- `boolean supportsTransactionIsolationLevel( int nivel)` throws `SQLException`
Devolve true si se soporta ese nivel de illamento nas transaccións.
- `boolean supportsDataDefinitionAndDataManipulationTransactions()` throws `SQLException`
Devolve true si se soportan sentenzas de manipulación de datos e de definición de datos dentro das transaccións.
- `boolean supportsDataManipulationTransactionsOnly()` throws `SQLException`
Devolve true si só se soportan sentenzas de manipulación de datos dentro das transaccións.

- boolean ***dataDefinitionCausesTransactionCommit()*** throws SQLException
Devolve true si as sentenzas de definición de datos provocan que se almacenen os resultados na base de datos (face un commit).
- boolean ***dataDefinitionIgnoredInTransactions()*** throws SQLException
Devolve true si se ignoran as sentenzas de definición de datos dentro das transaccións.

Métodos que determinan o soporte de características

Hai un gran grupo de métodos de `DatabaseMetaData` que permiten determinar si unha característica concreta (ou un conxunto concreto de características) está soportada polo conector.

Moitas destas características devolverán true ou false, dependendo de si a función dada é compatible ou non. Algúns destes métodos son:

- boolean ***supportsAlterTableWithAddColumn()*** throws SQLException
Devolve true si está soportada a orde ALTER TABLE con ADD COLUMN.
- boolean ***supportsAlterTableWithDropColumn()*** throws SQLException
Devolve true si está soportada a orde ALTER TABLE con DROP COLUMN.
- boolean ***supportsColumnAliasing()*** throws SQLException
Devolve true si se soportan os alias de columnas, podendo empregarse a palabra AS.
- boolean ***nullPlusNonNullIsNull()*** throws SQLException
Devolve true si o resultado de concatenar un NULL e un NOT NULL resulta NULL.
- boolean ***supportsConvert()*** throws SQLException
Devolve true si se soportan as conversións entre tipos de datos JDBC, é dicir, si soporta a función JDBC CONVERT.
- boolean ***supportsTableCorrelationNames()*** throws SQLException
Devolve true si se soportan os nomes de táboas correlacionadas.
- boolean ***supportsExpressionsInOrder()*** throws SQLException
Devolve true si se soportan expresións na cláusula ORDER BY.
- boolean ***supportsOrderByUnrelated ()*** throws SQLException
Devolve true si se soporta o uso dunha columna que non está na instrución SELECT nunha cláusula ORDER BY.
- boolean ***supportsGroupBy()*** throws SQLException
Devolve true si se soporta a cláusula GROUP BY.
- boolean ***supportsGroupByUnrelated()*** throws SQLException
Devolve true si se admite o uso dunha columna que non está na instrución SELECT nunha cláusula GROUP BY.
- boolean ***supportsGroupByBeyondSelect()*** throws SQLException
Devolve true si se admite o uso de columnas non incluídas na sentenza SELECT nunha cláusula GROUP BY, considerando que todas as columnas especificadas na sentenza SELECT se inclúen na cláusula GROUP BY.
- boolean ***supportsLikeEscapeClause()*** throws SQLException
Devolve true si se soportan as cláusulas LIKE.
- boolean ***supportsMultipleResultSets()*** throws SQLException
Devolve true si se soportan os resultados múltiples en sentenzas execute().
- boolean ***supportsMultipleTransactions()*** throws SQLException
Devolve true si se soportan transaccións simultáneas desde varias conexións.

- boolean ***supportsOuterJoins()*** throws SQLException
Devolve true si se soportan os outer joins.
- boolean ***supportsFullOuterJoins()*** throws SQLException
Devolve true si se soportan os outer full joins.
- boolean ***supportsLimitedOuterJoins()*** throws SQLException
Devolve true si existe soporte limitado para os outer joins.
- boolean ***supportsGetGeneratedKeys()*** throws SQLException
Devolve true si as claves xeradas automaticamente se poden recuperar logo dunha instrución se executou.
- boolean ***supportsStoredFunctionsUsingCallSyntax ()*** throws SQLException
Devolve true si se soporta a invocación das funcións definidas polo usuario ou do proveedor utilizando a sintaxe de chamada para procedementos almacenados.
- boolean ***supportsStoredProcedures()*** throws SQLException
Devolve true si se soportan chamadas a procedementos almacenados utilizando a sintaxe de escape proporcionada polo JDBC.
- boolean ***supportsSubqueriesInComparisons()*** throws SQLException
Devolve true si se soportan subconsultas nas expresións de comparación.
- boolean ***supportsSubqueriesInExists()*** throws SQLException
Devolve true si se soportan subconsultas EXISTS.
- boolean ***supportsSubqueriesInIns()*** throws SQLException
Devolve true si se soportan subconsultas nas expresións IN.
- boolean ***supportsSubqueriesInQuantifieds()*** throws SQLException
Devolve true si se soportan subconsultas nas expresións cuantificadas.

6.2 Interface ResultSetMetaData

Esta interface ofrécenos unha serie de métodos que nos permiten obter información sobre as columnas que contén o `ResultSet`.

É unha interface máis particular que a interface `DatabaseMetaData`. Un obxecto `DatabaseMetaData` é común para toda a base de datos, e cada obxecto `ResultSetMetaData` é particular para cada `ResultSet`.

O número de métodos que ofrece este interface é moito menor que o que ofrecía o interface `DatabaseMetaData`.

Un obxecto `ResultSetMetaData` obterémolo executando o método `getMetaData()` no obxecto `ResultSet` do que queiramos consultar a información das súas columnas, como se pode observar no seguinte código:

```
ResultSet rs=sentenza.executeQuery ();
ResultSetMetaData rsmd=rs.getMetaData ();
```

Métodos do ResultSetMetaData

Devolve o nome do catálogo.

- String ***getColumnClassName*** (int column)
Devolve o nome da clase Java á que corresponde o mapeo do tipo do campo.
- int ***getColumnCount*** ()
Devolve o número de columnas do obxecto `ResultSet`.

- **int *getColumnDisplaySize*** (int columnna)
Devolve o número máximo caracteres dunha columna.
- **String *getColumnLabel*** (int columnna)
Devolve o título da columna que se mostra nas saídas.
- **String *getColumnName*** (int columnna)
Obtén o nome da columna.
- **int *getColumnType*** (int columnna)
Obtén o tipo de datos SQL da columna.
- **String *getColumnTypeName*** (int columnna)
 - Devolve o tipo de datos JDBC que se corresponde co tipo da columna.
- **int *getPrecision*** (int columnna)
 - Devolve o número de díxitos decimais, nos datos de tipo numérico, o número máximo de caracteres para columnas de tipo carácter, e o número máximo de bytes para columnas de tipo binario.
- **int *getScale*** (int columnna)
 - Obtén o número de díxitos á dereita do punto decimal da columna.
- **String *getSchemaName*** (int columnna)
 - Obtén esquema da táboa da columna.
- **String *getTableName*** (int columnna)
 - Obtén o nome da táboa da columna.
- **boolean *IsAutoIncrement*** (int columnna)
 - Devolve true si a columna se incrementa automaticamente.
- **boolean *IsCaseSensitive*** (int columnna)
 - Devolve true si na columna se distingue entre maiúsculas e minúsculas.
- **boolean *isCurrency*** (int columnna)
 - Devolve true si a columna é un valor monetario.
- **boolean *isDefinitelyWritable*** (int columnna)
 - Devolve true si a escritura na columna se pode realizar con éxito.
- **int *isNullable*** (int columnna)
 - Indica si na columna se permiten valores nulos.
- **boolean *IsReadOnly*** (int columnna)
 - Devolve true si a columna é de só lectura.
- **boolean *isSearchable*** (int columnna)
 - Indica si a columna designada pode ser utilizada nunha cláusula where.
- **boolean *IsSigned*** (int columnna)
 - Indica si o valor almacenado nunha columna é un número con signo.
- **boolean *isWritable*** (int columnna)
 - Indica si é posible escribir na columna.

7. Xestión de Transaccións e procesamento por lotes

7.1 Xestión de transaccións

Nalgúns momentos podemos necesitar que varias sentenzas SQL se executen como un todo e si falla algunha delas se desfagan todos os cambios realizados ata o momento, volvendo á situación anterior á execución das sentenzas.

Por exemplo, ao realizar unha transferencia dunha conta dun banco a outra conta, as dúas sentenzas encargadas de realizar a actualización no saldo de cada conta débense executar, si unha delas falla deben desfacerse os cambios realizados. Estas dúas sentenzas débense executar nunha mesma transacción.

Unha transacción é unha unidade lóxica de traballo, é unha serie de sentenzas de actualización que deben completarse con éxito, antes de que os seus resultados se fagan permanentes na base de datos.

As transaccións deberían garantir todas as propiedades ACID: Atomicity, Consistency, Isolation and Durability (Atomicidade, Consistencia, Illamento e Durabilidade)

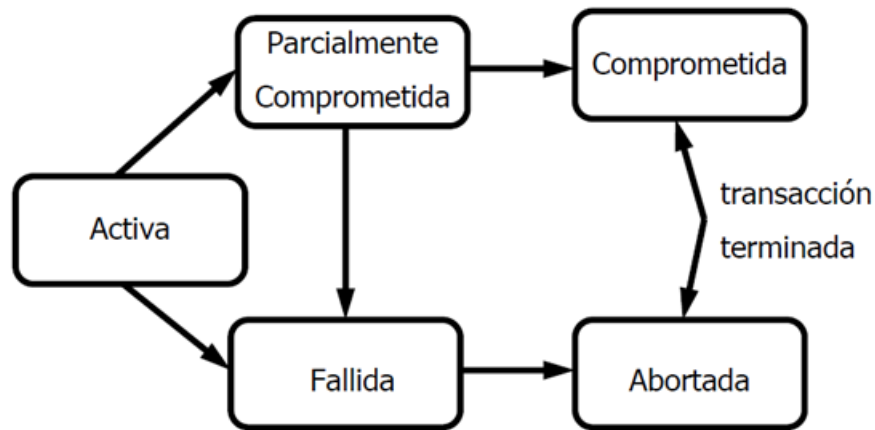
- **Atomicidade:** é a propiedade que asegura que a operación se realizou ou non, e polo tanto ante un fallo do sistema non pode quedar a medias. Se esta operación consiste nunha serie de pasos, todos eles acontecen ou ningún. Por exemplo, no caso dunha transacción bancaria ou se executa tanto o depósito e a dedución ou ningunha acción é realizada.
- **Consistencia** (Integridade): é a propiedade que asegura que só se empeza aquilo que se pode rematar, polo tanto execútanse aquelas operacións que non van romper as regras de integridade da base de datos. A propiedade de consistencia sostén que calquera transacción levará á base de datos dende un estado válido a outro tamén válido. A Integridade da Base de Datos permítenos asegurar que os datos son exactos e consistentes.
- **Illamento:** é a propiedade que asegura que unha operación non pode afectar a outras. Isto asegura que a realización de dúas transaccións sobre a mesma información sexan independentes e non xeren ningún tipo de erro. Esta propiedade define como e cando os cambios producidos por unha operación se fan visibles para as demais operacións concorrentes.
- **Durabilidade**(Persistencia): é a propiedade que asegura que unha vez realizada a operación, esta persistirá e non se poderá desfacer aínda que falle o sistema.

Si a transacción non se pode finalizar por calquera motivo, o SXBD garante que os cambios realizados por esta transacción son desfeitos.

Estados dunha transacción

Para garantir as propiedades ACID, as transaccións teñen que pasar por determinados estados:

- **Activa (Active):** é o estado inicial, a transacción permanece neste estado durante a súa execución.
- **Parcialmente comprometida (Uncommitted):** a transacción pasa a este estado cando acaba de realizar a última instrución.
- **Errada (Failed):** a transacción pasa a este estado tras descubrir que non se pode continuar a execución normal.
- **Abortada (RollBack):** a transacción pasa a este estado logo de retroceder a transacción e restablecer a base de datos ao estado que tiña cando comezou a transacción.
- **Comprometida (Committed):** a transacción pasa a este estado tras completarse con éxito.



O control das transaccións polas aplicacións supón a confirmación ou retroceso das operacións para deixar a base de datos nun estado consistente. Este control realízase mediante dúas sentenzas:

- **commit:** sinala o final con éxito dunha transacción. Os cambios fanse permanentes.
- **rollback:** sinala o final sen éxito dunha transacción. O SXBD debe retroceder e desfacer os cambios realizados.

Modos de confirmación de transaccións

As transaccións en JDBC *adminístranse nas conexións*. Cando unha aplicación completa unha transacción, ten que confirmar ou revertir todo o traballo completado e realízao a través dos métodos que nos brinda dita conexión.

JDBC soporta dous modos de administracións de transaccións:

- **Modo de confirmación automática.**

Si unha conexión está en modo de confirmación automática, todas as súas instrucións SQL execútanse e confírmanse como transaccións individuais, é dicir, cada sentenza SQL é tratada como unha transacción, e faise efectiva cando se complete a sentenza.

Por defecto, todas as conexións que obtemos coa base de datos son "autocommit". Isto significa que cada instrución SQL individual é tratada como unha transacción e será automaticamente confirmada e executada. Non se require ningunha outra función de administración de transaccións.

- **Modo de confirmación manual.**

Si unha conexión está en modo de confirmación manual, as súas instrucións SQL agrúpanse en transaccións que finalizan facendo unha chamada explícita ao método `commit` ou ao método `rollback` do obxecto `Connection`.

Cando múltiples instrucións son executadas nunha única transacción, todas as operacións son realizadas (convertidas en permanentes na base de datos) ou descartadas (desfáanse os cambios aplicados á base de datos).

É *responsabilidade do programador confirmar ou revertir as transaccións*. Si non se chama ao método `commit()`, os cambios non serán gardados na base de datos aínda cando se teñan executadas unha ou máis sentenzas SQL da transacción.

Cambiar o modo de confirmación da conexión

Para cambiar o modo de confirmación, utilízase o método `setAutoCommit()` da interface `Connection`. A sintaxe deste método é :

```
void setAutoCommit(boolean confirmación) throws SQLException
```

- Modo confirmación manual: `conexion.setAutoCommit(false);`
- Modo confirmación automática: `conexion.setAutoCommit(true);`

Ver o estado do modo de confirmación da conexión

Para ver o actual modo de confirmación da conexión, utilízase o método `getAutoCommit ()` da interface `Connection`. A sintaxe deste método é:

```
Boolean getAutoCommit() throws SQLException
```

`true`: modo de confirmación automática

`false`: modo de confirmación manual

Iniciar unha transacción

No procesamento de transaccións, en xeral, podemos ter tres xeitos de controlar cando se inicia unha transacción:

- **Transaccións explícitas:** Comezan explicitamente coa sentenza `BEGIN TRANSACTION`. Esta maneira non é soportada polas conexións JDBC.
- **Transaccións de confirmación automática:** Cada instrución SQL comeza automaticamente unha transacción e confírmase cando termina. Non se ten que especificar ningunha sentenza adicional para controlar as transaccións. É a maneira por defecto das conexións JDBC.
- **Transaccións implícitas:** Unha transacción SQL comeza automaticamente coa primeira sentenza SQL ata confirmar ou reverter esta transacción, o cal inicia automaticamente unha nova transacción. Todas as sentenzas SQL executadas tras a última chamada a un método `commit` están incluídas na transacción actual e son tratadas xuntas como unha unidade. Non se ten que realizar ningunha acción para delinear o inicio dunha transacción. O modo de transaccións implícitas xera unha cadea continua de transaccións, é dicir, cada sentenza comeza automaticamente unha nova transacción e só finalizan todas cando se executa unha sentenza `COMMIT`(confirmar) ou `ROLLBACK` (reverter).

As conexións JDBC inician as transaccións implícitas cando se desactiva o modo de confirmación automática (`setAutoCommit(false)`).

Confirmar ou desfacer os cambios da transacción

Nas transaccións implícitas, para facer permanentes os cambios realizados na base de datos pola transacción actual ou desfacerlos, a API JDBC inclúe dous métodos como parte do interface `Connection`:

```
void commit () throws SQLException
void rollback () throws SQLException
```

`commit`: fai permanentes todos os cambios realizados desde o último `commit` ou `rollback` e libera calquera bloqueo de base de datos actualmente en poder da conexión.

`rollback`: desfai todos os cambios realizados na transacción actual e libera os bloqueos de base de datos actualmente en poder da conexión.

Estes métodos deben ser utilizados só cando o modo de confirmación automática se desactivou.

Erros ao procesar a transacción

Nas transaccións implícitas, si un erro impide a terminación correcta dunha transacción, hai que reverter a transacción e liberar todos os recursos que mantén a transacción. No caso contrario, hai que facer permanentes os cambios.

Si ocorre algún erro na transacción, lánzase a excepción `SQLExceptions`. O control de excepcións realízase envolvendo no `try...catch` os métodos `commit` e `rollback`. Na parte do `try`, ponse o `commit` para terminar correctamente a transacción e na do `catch`, ponse o `rollback` para controlar os erros e reverter a transacción.

```
try{
    .....
    commit
}catch (SQLException ex){
    .....
    rollback
}
```

Exemplo de xestión de transaccións

As transaccións van permitir asegurar que un conxunto de instrucións SQL sexan executadas con éxito cada unha delas ou que ningunha sexa executada. Por exemplo, a transferencia dun importe de diñeiro entre dúas contas bancarias representa un claro exemplo da necesidade de utilizar unha transacción. Si dunha conta sacamos diñeiro para ingresalo noutra conta, as dúas operacións débense levar a cabo ou si ocorre un erro, ningunha delas.

Como exemplo, imos realizar un método que realice a transferencia de diñeiro entre as dúas contas e ademais que se imprima un informe cando se realiza unha operación nunha conta; primeiro, imos facelo sen control de transaccións e despois con control de transaccións para ver a diferenza.

■ Sin control de transaccións

```
public void movimiento (Connection con, String contaDebito, String contaCredito,
double importe) throws SQLException
{
    PreparedStatement sentenza;
    try {
        sentenza = con.prepareStatement("update conta set saldo = saldo +?
                                     where numeroconta =?");
        sentenza.setDouble(1, importe*-1);
        sentenza.setString(2, contaDebito);
        sentenza.executeUpdate();
        imprimirInforme(contaDebito, importe);
        sentenza.setDouble(1, importe);
        sentenza.setString(2, contaCredito);
        sentenza.executeUpdate();
        imprimirInforme(contaCredito, importe);
    } catch (SQLException ex) {
        System.out.println("SQLException: " + ex.getMessage());
    } finally {
        if (senteza != null) {
            sentenza.close();
        }
    }
}
```

No anterior código, a transacción é confirmada automaticamente, polo que cada sentenza de actualización é unha transacción que será confirmada cada unha por separado cando termina. Si ocorre un erro no medio, por exemplo, si o método `imprimirInforme` activa unha excepción, a execución continúa pola instrución que segue ao bloque `catch`. As instrucións que hai desde onde ocorreu o erro ata o final do bloque `try`, non se executan. Neste caso descontarase o diñeiro da conta de débito, pero non se ingresará na conta de crédito.

Unha solución é controlar manualmente a confirmación das transaccións e utilizar `commit` ou `rollback` para confirmar ou reverter as operacións, de tal xeito, que se efectúen as dúas operacións ou ningunha.

■ Con control de transaccións

```
public void movimiento (Connection con, String contaDebito, String contaCredito,
double importe )throws SQLException {
    PreparedStatement sentenza;
    try {
        sentenza = con.prepareStatement("update conta set saldo = saldo + ?
                                     where numeroconta = ?");
        con.setAutoCommit(false);
        sentenza.setDouble(1, importe*-1);
        sentenza.setString(2, contaDebito);
        sentenza.executeUpdate();
        imprimirInforme(contaDebito, importe);
        sentenza.setDouble(1, importe);
        sentenza.setString(2, contaCredito);
        sentenza.executeUpdate();
        imprimirInforme(contaCredito, importe);
        con.commit();
    } catch (SQLException ex) {
        System.out.println("SQLException: " + ex.getMessage());
        if (con != null)
            con.rollback();
    }
}
```

```

try {
    System.out.println("A transacción foi abortada ");
    con.rollback();
} catch(SQLException e) {
    System.out.print("Ocorreu un erro ");
} finally {
    if (sentenza != null) {
        sentenza.close();
    }
    con.setAutoCommit(true);
}
}

```

Neste caso, si o método `imprimirInforme` activa unha excepción, a execución continúa no bloque `catch` e realiza un `rollback`, desfazendo as operacións anteriores, non se desconta o diñeiro da conta. Polo contrario, si chega a executarse a sentenza `commit`, a transacción termina con éxito e fai permanente as dúas operacións na base de datos, a transferencia efectúase.

Transaccións e acceso concorrente

Nun contorno monousuario, as transaccións están illadas unhas das outras e son bastantes sinxelas de manexar, simplemente implican salvar ou desfacer os cambios. En cambio, nun entorno multiusuario, as transaccións son máis complexas. O problema fundamental é que poden producirse problemas de simultaneidade. Por exemplo, si varios usuarios intentan ter acceso a unha base de datos ao mesmo tempo, as súas transaccións poderían tratar de realizar operacións sobre os mesmos datos ao mesmo tempo, interferindo unhas con outras. Isto pode producir datos erróneos, perda de operacións, violación de regras de integridade, chegando a estar a base de datos nun estado de inconsistencia.

Razóns para permitir a concorrencia

Aumentar a produtividade: número de transaccións executadas por minuto.

Reducir o tempo medio de resposta de transaccións (as transaccións pequenas non esperan ás grandes).

Problemas no acceso concorrente

Existen tres problemas comúns nos que unha transacción, aínda que sexa correcta por si mesma, pode producir unha resposta incorrecta, si algunha outra transacción que tamén é correcta, interfere sen control con ela accedendo aos mesmos datos.

■ Actualización perdida (lost update)

Unhas transaccións sobrescriben as actualizacións doutras.

Ocorre cando unha transacción T1 modifica datos que foron modificados por outra transacción T2, co cal, as modificacións de T2 pérdense.

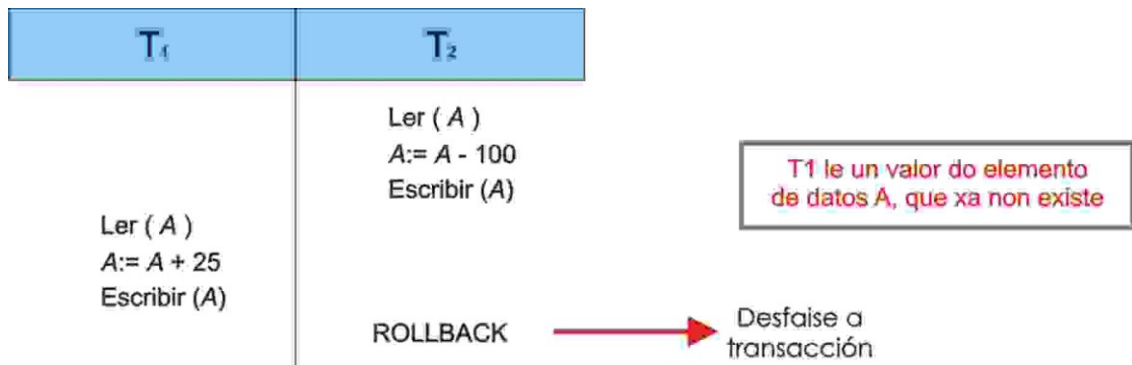
Transacción 1	Transacción 2
$x_1 = \text{LER}(X);$	
	$x_2 = \text{LER}(X);$ $x_2 = x_2 + 1;$ $\text{ESCRIBIR}(X, x_2);$ $\text{COMMIT};$
$x_1 = x_1 + 1;$ $\text{ESCRIBIR}(X, x_1);$ $\text{COMMIT};$	



■ Lecturas sucias (dirty reads)

Lese un valor que está sendo modificado por outra transacción que non finalizou e que podería cancelarse ou fallar.

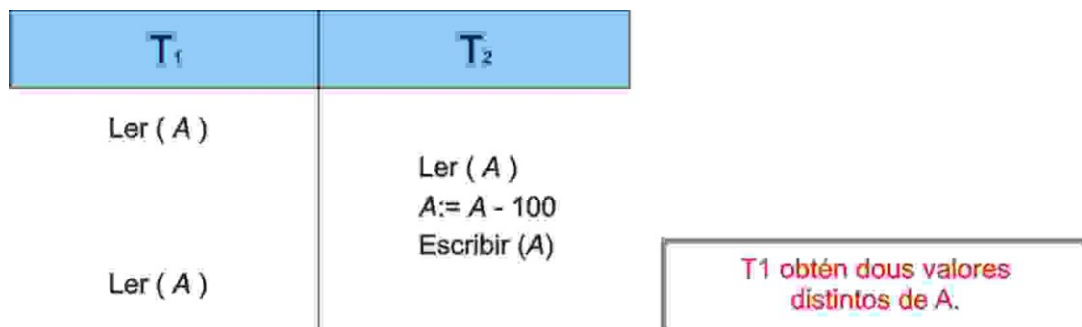
Unha lectura sucia ocorre cando unha transacción T1 usa datos que foron modificados por outra transacción T2, e estes datos están nun estado sen confirmar. Polo tanto, existe a posibilidade de que T2 desfaga os cambios. Neste caso, T1 estará utilizando uns datos que xa non existen (datos sucios). T1 opera sobre unha suposición falsa.



■ Lecturas non-repetibles ou difusas (non repeatable reads)

Obtéñense lecturas diferentes do mesmo valor durante a mesma transacción.

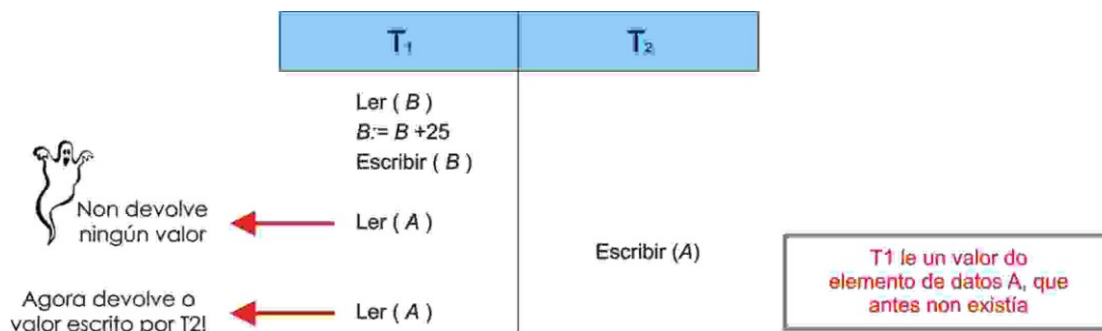
Unha lectura non-repetible prodúcese cando unha transacción T1 volve ler un elemento de datos que xa lera previamente, pero que foi modificado por outra transacción. Así, a transacción T1 estará lendo dous valores distintos para o mesmo elemento de datos.



■ Lecturas pantasma (phantom reads)

Obtéñense lecturas diferentes dun conxunto de tuplas (filas) durante a mesma transacción.

Unha lectura fantasma ocorre cando unha transacción adquire datos mediante algunha consulta, outra transacción modifica algúns dos datos, e a transacción orixinal recupera os datos unha segunda vez. A primeira transacción agora terá un conxunto de resultados diferentes, que poderían conter datos pantasma.



Técnicas de control de concurrencia. Bloqueos

Os SXBD multiusuarios teñen mecanismos para o control da concurrencia e manter a consistencia da base de datos cando esta é actualizada por múltiples usuarios.

Unha das técnicas principais é utilizar *bloqueos (locks)* para controlar o acceso concorrente aos datos.

Os bloqueos son mecanismos para sincronizar o acceso aos datos dunha base de datos por parte de transaccións concorrentes.

■ Un bloqueo

- ✓ É unha información asociada a cada elemento de datos que describe o estado do elemento respecto as posibles operacións (recuperación ou actualización) que se poden realizar sobre el en cada momento.
- ✓ O SXBD impón os bloqueos necesarios en cada momento.

■ Os elementos de datos

- ✓ Son as unidades de datos para os que se controla o acceso.
- ✓ Hai varios niveis: táboa, fila, de rangos de filas, columna, campos, clave, base de datos, etc.

■ A granularidade

É o tamaño dos elementos. Así fálase de sistemas de gran fino ou de gran groso para denotar elementos pequenos ou grandes respectivamente.

A maior granularidade, menor concurrencia.

- Granularidade moi grosa implica xestionar menor números de bloqueos, pero atrasa a execución de moitas transaccións (os obxectos non se van liberando).
- Unha granularidade moi fina, permite maior concurrencia, pero aparecen mais situacións de bloqueos mutuos (deadlock) que terá que ser resoltas.

■ Bloqueos mutuos ou deadlock

- ✓ Prodúcese por transaccións que acceden a múltiples táboas e se están bloqueando mutuamente entre elas.
- ✓ Por exemplo: a transacción 1 require inserir dato na táboa A e actualizar a táboa B. A transacción 2 require cambiar a táboa B e logo actualizar a táboa A. A transacción 1 non pode terminar porque a transacción 2 ten bloqueada a táboa B e a transacción 2 non pode terminar porque a transacción 1 ten bloqueada a táboa A.

Tipos de bloqueo:

■ Bloqueo compartido (de lectura):

- ✓ Si unha transacción ten un bloqueo compartido sobre un elemento de datos, *pode ler o elemento, pero non actualízalo* (escribir).
- ✓ Varias transaccións poden manter á vez bloqueos compartidos sobre o mesmo elemento.
- ✓ Este tipo utilízase cando non se require actualizar datos, pero deséxase impedir calquera modificación mentres os datos son consultados.

■ Bloqueo exclusivo (de escritura):

- ✓ Si unha transacción ten un bloqueo exclusivo sobre un elemento de datos, *pode ler e actualizar* (escribir) o elemento.
- ✓ Un bloqueo exclusivo proporciona acceso exclusivo ao elemento. Ningunha outra transacción pode acceder ao obxecto bloqueado, nin bloquealo ata que sexa liberado pola transacción que o bloquea.
- ✓ Utilízase cando se quere actualizar datos.

O nivel de illamento (transaction isolation level)

En bases de datos, o illamento é unha propiedade ACID que define como e cando os cambios producidos por unha operación se fan visibles para as demais operacións concorrentes.

O nivel de illamento resolve problemas asociados a múltiples transaccións de diferentes usuarios solicitando os mesmos datos.

Define o grado en que se illa unha transacción das modificacións de recursos ou datos realizadas por outras transaccións.

O nivel de illamento dunha transacción é *unha característica de vital importancia* no desenvolvemento de aplicacións de base de datos, xa que afecta aos tipos e duración de bloqueos que se producen na base de datos, o que ten efectos directos no rendemento e tempo de resposta das nosas consultas e transaccións. Evidentemente, a elección do modo de illamento é máis importante canto maior é a concorrencia da base de datos.

A maior nivel de illamento garante unha maior consistencia nos datos, pero a costa dunha menor concorrencia e menor rendemento.

SQL-92 define catro niveles de illamento dentro dunha transacción:

■ **Serializable**

- ✓ Este é o nivel de illamento máis alto. Especifica que todas as transaccións ocorran de modo illado, é dicir, coma se todas as transaccións se executasen en modo serie (unha tras outra).
- ✓ Garante que unha transacción recuperará exactamente os mesmos datos cada vez que repita unha operación de lectura. Por exemplo, a mesma sentenza SELECT coa mesma cláusula WHERE devolverá o mesmo número de filas, logo non se poderán inserir filas novas no rango cuberto pola WHERE, e evítanse as lecturas fantasma.
- ✓ O SXBD aplicará un nivel de bloqueo con granularidade grosa, o que pode afectar aos demais usuarios, nos sistemas multiusuario.
- ✓ Ofrece unha maior consistencia na transacción utilizando máis recursos e bloqueos.
- ✓ Evita os problemas das lecturas sucias (dirty reads), das lecturas non repetibles (non repeatable reads), e das lecturas fantasma (phantom reads).
- ✓ Vantaxes: consistencia de datos absoluta.
- ✓ Contras: altísimo grado de bloqueo e moi baixo nivel de concorrencia, xa que as táboas poden ser bloqueadas na súa totalidade mentres se realiza unha transacción.
- ✓ Uso habitual: non é moi habitual. Só en casos moi raros cando se requira unha gran consistencia de datos.

■ **Lecturas repetibles (REPEATABLE READ)**

- ✓ Evita que entre dúas lecturas dun mesmo rexistro nunha transacción A, outra transacción B poida modificar o devandito rexistro, co efecto de que na segunda lectura da transacción A se obtivese un dato diferente. Deste modo, ambas as dúas lecturas serán iguais (lecturas repetidas).
- ✓ Evítanse os problemas das lecturas sucias e das lecturas non repetibles.
- ✓ Non obstante, non evita as lecturas fantasma, é dicir, unha transacción podería executar unha consulta sobre un rango de filas (por exemplo: 100 filas) e de forma simultánea outra transacción podería realizar un inserción dunha ou varias filas sobre o mesmo rango.
- ✓ Vantaxes: alta consistencia nos datos.
- ✓ Contras: maior nivel de bloqueos e diminución do grado de concorrencia.
- ✓ Uso habitual: En operacións bancarias ou financeiras que requiran un grado de exactitude moi alto.

■ **Lecturas comprometidas (READ COMMITTED)**

- ✓ Só permite ler datos confirmados antes de que se execute unha consulta.
- ✓ Evita as lecturas sucias (dirty reads), pero pola contra permite lecturas non repetibles e lecturas fantasma. Non hai bloqueos de rango.
- ✓ Vantaxes: bo equilibrio entre concorrencia e consistencia.
- ✓ Contras: maior nivel de bloqueo.
- ✓ Uso habitual: é o isolation level máis común, de feito o que se *usa por defecto en numerosos SXBD*.

■ Lecturas non comprometidas (READ UNCOMMITTED)

- ✓ Este é o menor nivel de illamento. Permite ler datos non confirmados doutras transaccións.
- ✓ Neste nivel pódense producir todos os efectos secundarios de simultaneidade (lecturas sucias, lecturas non repetibles e lecturas fantasma).
- ✓ Vantaxes: non se necesita facer bloqueos para ler datos, aínda que si para poder modificalos. Ofrece grandes beneficios de rendemento.
- ✓ Contras: non se pode garantir a consistencia dos datos lidos. Pode darse o caso de que se lea un dato que logo non exista máis.
- ✓ Uso habitual: en situacións de alta concorrencia onde a potencial inconsistencia de datos non sexa un problema.

O nivel de illamento apropiado depende do equilibrio entre os requisitos de integridade dos datos da aplicación e a sobrecarga de cada nivel de illamento.

■ Nivel de illamento menor

- ✓ Significa que moitos usuarios poden ter acceso aos datos simultaneamente, pero tamén aumentan os efectos de simultaneidade que poden experimentar, como lecturas non confirmadas ou perda de actualizacións.

■ Nivel de illamento maior

- ✓ Reduce os tipos de efectos de simultaneidade, pero require máis recursos do sistema e aumenta as posibilidades de que unha transacción bloquee a outra.

O conector JDBC e o nivel de illamento

Normalmente, non é necesario facer nada sobre o nivel de illamento da transacción, simplemente usar o que hai por defecto para o SXBD.

JDBC permite descubrir cal é o nivel de illamento de transacción que está definido para o SXBD e tamén permite establecelo a outro nivel.

A interface `Connection` proporciona as seguintes constantes para os niveis de illamento:

■ `static int TRANSACTION_NONE`

- ✓ Indica que as transaccións non son compatíbeis.

■ `static int TRANSACTION_READ_UNCOMMITTED`

- ✓ Indica que as lecturas sucias, as lecturas non repetibles e as lecturas fantasma poden ocorrer. É o nivel 1.

■ `static int TRANSACTION_READ_COMMITTED`

- ✓ Indica que se impiden lecturas sucias; as lecturas non repetibles e as lecturas fantasma poden ocorrer. É o nivel 2.

■ `static int TRANSACTION_REPEATABLE_READ`

- ✓ Indica que se impiden as lecturas sucias e lecturas non repetibles; as lecturas fantasma poden ocorrer. É o nivel 3.

■ `static int TRANSACTION_SERIALIZABLE`

- ✓ Indica que se impiden as lecturas sucias, as lecturas non repetibles e as lecturas fantasma. É o nivel 4.

Obter o nivel de illamento actual dunha conexión

Para obter o nivel de illamento dunha transacción, úsase o seguinte método da interface `Connection`:

```
int getTransactionIsolation () throws SQLException
```

Devolve o nivel de illamento actual, que será unha das constantes de nivel de illamento. Exemplo:

```
if (conexion.getTransactionIsolation() == TRANSACTION_SERIALIZABLE)
```


Cambiar o nivel de illamento dunha conexión

Para cambiar o nivel de illamento dunha transacción úsase o seguinte método da interface `Connection`:

```
void setTransactionIsolation (int nivel) throws SQLException
```

As constantes definidas na interface de conexión son os posibles niveis de illamento de transacción. Exemplo:

```
Connection.setTransactionLevel (TRANSACTION_SERIALIZABLE);
```

Un conector JDBC *pode non soportar todos os niveis de illamento de transacción*. Se un conector non soporta o nivel de illamento especificado nunha invocación do método `setTransactionIsolation`, o conector pode substituílo por un superior, cun nivel de illamento máis restritivo. Si un conector non pode substituír a un maior nivel de transacción, lanza unha excepción `SQLException`.

Determinar si un nivel de illamento é soportado por un conector JDBC

Para determinar si un conector soporta ou non un determinado nivel, úsase o método `supportsTransactionIsolationLevel` da interface `DatabaseMetaData`.

Sintaxe:

```
Boolean supportsTransactionIsolationLevel(int nivel) throws SQLException
```

Onde o parámetro `nivel` será unha das constantes de nivel de illamento da interface `Connection`.

Exemplo:

```
DatabaseMetaData metadatos = conexion.getMetaData();
if(metadatos.supportsTransactionIsolationLevel(
    Conexion.TRANSACTION_READ_COMMITTED))
{
    System.out.println("nivel de illamento: "+
        "TRANSACTION_READ_COMMITTED é soportado.");
    conexion.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
}
```

7.1.1 Puntos de retorno (Savepoints)

Os puntos de retorno (savepoints) ofrecen un mecanismo para reverter partes dunha transacción, é dicir, permite ir gardando estados da transacción que son válidos, para que en caso de fallos non se teña que abortar toda a transacción, senón ata certos puntos (savepoints).

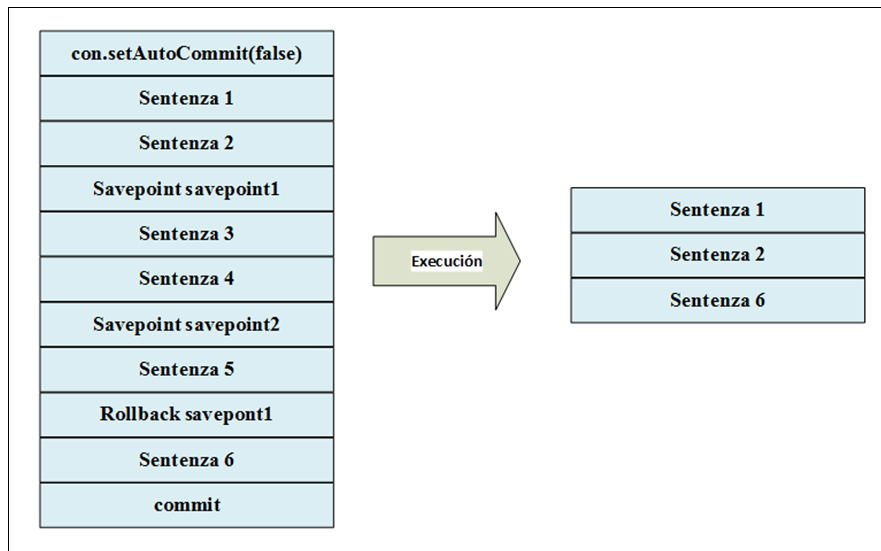
Cando creamos un punto de retorno, indicamos un punto dentro dunha transacción ao que se pode facer `rollback`, sen afectar a calquera traballo realizado na transacción antes de que o punto de retorno fose creado. Podemos crear varios puntos de retorno dentro dunha transacción individual.

Os puntos de retorno poden ser útiles para implementar a recuperación de erros complexos en aplicacións de base de datos. Si ocorre un erro no medio dunha transacción de múltiples sentenzas, a aplicación pode ser capaz de recuperarse do erro (devolvendo `rollback` ata un savepoint) sen necesidade de abortar a transacción completa.

Os puntos de retorno están definidos no estándar SQL e sopórtano a maioría dos SXBDS máis utilizados, como: Oracle, PostgreSQL, Microsoft SQL Server, MySQL, DB2 e Firebird.

Cando se establece un punto de retorno defínese un punto de reversión lóxico dentro dunha transacción. Si ocorre un erro máis aló dun punto de retorno, pode utilizar o método `rollback` para desfacer ou ben todos os cambios, ou só os cambios realizados tralo punto.

Na seguinte figura, móstrase un exemplo de utilización de puntos de retorno. Ao iniciar a transacción as sentenzas 1 e 2 son protexidas polo "savepoint1", indicando que este bloque de instrucións pode conservarse no caso dunha cancelación parcial; non obstante as sentenzas 3,4,5 son canceladas debido á sentenza "rollback savepoint1", que indica que se cancelen todas as operacións realizadas despois do savepoint1. Neste caso, só se executarán as sentenzas 1,3 e 6 cando se confirme a transacción.



O obxecto `Connection` dispón de dous novos métodos que axudan a administrar os puntos de rescate.

Establecer un punto de retorno

Para establecer un punto de retorno e desfacer os cambios despois dese punto de retorno, úsase o método `setSavepoint` da interface `Connection`. A sintaxe é:

```
Savepoint setSavepoint (String punto_retorno) throws SQLException
```

Este método devolve un obxecto `Savepoint` que o representa ao punto de retorno.

Si `setSavepoint` se invoca fóra dunha transacción activa, iniciárase unha transacción neste punto de retorno.

Eliminar un punto de retorno.

Para quitar un punto de retorno da transacción actual, úsase o método `releaseSavepoint` da interface `Connection`. Libéranse os recursos ocupados. A sintaxe é:

```
void releaseSavepoint (Savepoint punto_retorno) throws SQLException
```

Unha referencia a un punto de retorno eliminado, provocará unha excepción `SQLException`. Tamén se pode lanzar a excepción `SQLFeatureNotSupportedException`, si o controlador `JDBC` non é compatible con este método.

Calquera punto de retorno creado nunha transacción é automaticamente eliminado cando a transacción se confirma ou se reverte.

Desfacer os cambios a partir dun punto de retorno

Podemos pasar un punto de retorno como argumento, ao método `rollback` da interface `Connection` para desfacer todas as sentenzas ata o punto de retorno especificado. A sintaxe é:

```
void rollback (Savepoint punto_retorno) throws SQLException
```

Ao revertir unha transacción a un punto de retorno, elimínanse automaticamente o resto dos puntos creados despois deste punto de retorno.

Determinar si un conector JDBC soporta puntos de retorno.

Para determinar si un conector soporta ou non un determinado nivel, úsase o método `supportsSavepoints()` da interface `DatabaseMetaData`.

```
Boolean supportsSavepoints() throws SQLException
```

Devolve `true` si o conector `JDBC` soporta puntos de retorno; `false` pola contra.

Exemplo

No seguinte método, úsase unha transacción para realizar varias insercións na táboa Animal e créase un punto de retorno para protexer a realización das dúas primeiras instrucións. Como resultado da execución do método, só se confirman na base de datos as dúas primeiras instrucións, xa que se desfán todas as operacións despois do punto de retorno. No caso de que unha instrución viole algunha restrición, lanzaríase unha excepción executando a instrución `rollback`, polo que non se inseriría ningún rexistro na base de datos.

```
public static void exTransaction(Connection con) {
    try {
        con.setAutoCommit(false);
        Statement stmt = con.createStatement();
        stmt.executeUpdate("INSERT INTO Animal(id, nome) VALUES(1, 'Rocky');");
        stmt.executeUpdate("INSERT INTO Animal(id, nome) VALUES(2, 'Miau');");
        Savepoint puntol = con.setSavepoint();
        stmt.executeUpdate("INSERT INTO Animal(id, nome) VALUES(3, 'Sansón');");
        con.rollback(puntol);
        con.commit();
        stmt.close();
        System.out.println("Transacción realizada.");
    }
    catch (SQLException ex) {
        ex.printStackTrace();
        try {
            System.out.println("Transacción falida.");
            con.rollback();
        }
        catch (SQLException se) {
            se.printStackTrace();
        }
    }
}
```

7.2 Procesamento por lotes e transaccións

Para aumentar o rendemento ao realizar varias actualizacións nunha base de datos e reducir a cantidade de sobrecarga de comunicación, o conector JDBC ofrece a posibilidade de enviar varias actualizacións como unha soa unidade de traballo, denominada tamén lote. Esta operación é moito máis eficiente que realizar as actualizacións de xeito separado.

As operacións por lotes foron introducidas en JDBC 2.0 e requiren que *unha transacción non estea en modo autocommit*.

As interfaces `Statement`, `PreparedStatement` e `CallableStatement` pódense usar para enviar actualizacións por lotes.

Engadir unha sentenza á lista do lote

A sintaxe do método para engadir unha sentenza á lista de comandos para ser executadas como un lote é:

```
void addBatch (String sql) throws SQLException
```

O procesamento por lotes só pode ser utilizado con sentenzas que *devolvan un valor numérico*, pero non con sentenzas que devolvan un conxunto de resultados. Isto quere dicir, que só pode ser utilizado con sentenzas do tipo: `INSERT INTO`, `UPDATE`, `DELETE`, `CREATE TABLE`, `DROP TABLE` e `ALTER TABLE`.

Executar unha lista de sentenzas

Para enviar un lote de sentenzas á base de datos para que se execute, úsase o seguinte comando:

```
int [] executeBatch () throws SQLException
```

As sentenzas do lote execútanse en serie, na orde en que se engadiron ao lote.

Si todas as sentenzas se executan correctamente, devolve un array de enteiros que contén unha entrada para cada elemento no lote. Cada elemento do array é un contador de actualizacións (o número de filas afectadas en cada actualización). As entradas do array ordénanse de acordo coa orde en que se procesan as sentenzas. Por exemplo, se catro sentenzas se executan satisfactoriamente, o array conterá catro números: o primeiro é o contador da primeira sentenza, o segundo da segunda sentenza e así sucesivamente.

Exemplo de execución dun lote:

```
Statement stmt = null;
try {
    conexion.setAutoCommit(false);
    stmt = conexion.createStatement();
    stmt.addBatch("DELETE FROM Animal");
    stmt.addBatch("INSERT INTO Animal(id, nome) VALUES(1, 'Martín)");
    stmt.addBatch("INSERT INTO Animal(id, nome) VALUES(2, 'Alex')");
    stmt.addBatch("INSERT INTO Animal(id, nome) VALUES(3, 'Norman')");
    int[] contador = stmt.executeBatch();
    conexion.commit();
} catch (BatchUpdateException be) {
}
} catch (Exception e) {
}
```

Manexo de excepcións na actualización Batch

Existen dúas excepcións que poden ser lanzadas durante unha actualización por lotes: `SQLException` e `BatchUpdateException`.

Todos os métodos do API JDBC lanzarán un obxecto `SQLException` si existe algún problema de acceso á base de datos. Si unha sentenza nun proceso por lotes non se executa correctamente, este método produce unha `BatchUpdateException`.

Si ao chamar ao método `executeBatch`, unha das instrucións que engadimos ao batch produce un `ResultSet` ou unha das instrucións do batch non se executa satisfactoriamente por algunha outra razón, lanzarase unha excepción `BatchUpdateException`,

```
public class BatchUpdateException extends SQLException
```

Ademais da información proporcionada por `SQLException` da que herda `BatchUpdateException`, proporciona os contadores de actualización para todas as sentenzas que se executaron con éxito durante a actualización (similar ao array retornado polo método `executeBatch`).

Ao lanzarse unha excepción `BatchUpdateException` por que unha sentenza do lote falla, o controlador JDBC pode ou non continuar procesando os comandos restantes do lote, dependendo de como procese o SXBD en particular estes lotes.

Si o controlador segue co procesamento logo dun fallo, o array de contadores de actualización devolto polo método `BatchUpdateException` conterá tantos elementos como sentenzas hai no lote, e os valores poden ser os seguintes:

- Un número maior ou igual a cero: indica que o mandato se procesou con éxito, e o valor indica o número de filas que se viron afectadas pola execución da sentenza na base de datos.
- indica que o mandato se procesou con éxito e será o número de filas na base de datos que se viron afectadas pola execución da sentenza.
- Un valor de `SUCCESS_NON_INFO`: indica que a sentenza se procesou con éxito, pero que o número de filas afectadas é descoñecido.
- Un valor de `EXECUTE_FAILED`: indica que o comando non se executou con éxito.

Podemos utilizar o seguinte método para obter o *array de contadores de actualizacións* do obxecto `BatchUpdateException`.

```
int [] getUpdateCounts ()
```

Exemplo: No seguinte fragmento de código execútase un proceso por lotes. Si ocorre un fallo imprímense os contadores de actualización contidos no obxecto BatchUpdateException.

```
Statement stmt = null;
try {
    conexion.setAutoCommit(false);
    stmt = conn.createStatement();
    stmt.addBatch("DELETE FROM Animal");
    stmt.addBatch("INSERT INTO Animal(id, nome) VALUES(1, 'Martín)");
    stmt.addBatch("INSERT INTO Animal(id, nome) VALUES(2, 'Alex')");
    stmt.addBatch("INSERT INTO Animal(id, nome) VALUES(3, 'Norman')");
    int[] contadorAct = stmt.executeBatch();
    conexion.commit();

} catch (BatchUpdateException e) {
    int[] contadorAct = e.getUpdateCounts();
    verificarContador(contadorAct);
    try {
        conexion.rollback();
    } catch (Exception e2) {
        e.printStackTrace();
    }
} catch (Exception e) {
    e.printStackTrace();
} finally {
    stmt.close();
}

public static void verificarContador(int[] contador)
{
    for (int i = 0; i < contador.length; i++)
    {
        if (contador[i] >= 0)
        {
            System.out.println("Sentenza executada con éxito;
                                contador=" + contador[i]);
        } else if (contador[i] == Statement.SUCCESS_NO_INFO)
        {
            System.out.println("Sentenza executada con éxito;
                                contador=Statement.SUCCESS_NO_INFO");
        } else if (contador[i] == Statement.EXECUTE_FAILED) {
            System.out.println("sentenza falida;
                                contador=Statement.EXECUTE_FAILED");
        }
    }
}
```

Borrar a lista de sentenzas do lote

Para eliminar todas as sentenzas que se engadan co método addBatch, úsase o seguinte comando:

```
void clearBatch() throws SQLException
```

Non se pode elixir selectivamente que sentenzas quitar.

Ver si o conector JDBC é compatible con procesos por lotes

Non todos os conectadores JDBC soportan o procesamento por lotes. Podemos utilizar o método supportsBatchUpdates da interfaz DatabaseMetaData para determinar si SXBD admite o procesamento por lotes. Sintaxe:

```
boolean supportsBatchUpdates() throws SQLException
```

O método devolve true si o controlador JDBC é compatible con esta característica.